

FACULTY PROFILE



PARTICULARS	DETAILS	
Name	Rupesh Kumar	
Designation	Lecturer	
Date Of Birth	11/03/1990	
Branch	INFORMATION TECHNOLOGY	
AICTE Unique Id	1-4397574488	
Education Qualification	B.Tech(IT)	
Work Experience:	Teaching	9 Yrs
	Research	NIL
	Industry	NIL
	Others	NIL
Area of Specialization	INFORMATION TECHNOLOGY	
Courses taught (Please Tick)	Diploma	✓
	Post Diploma	NIL
	Under Graduate	NIL
	Post Graduate	NIL
	Post Graduate Diploma	NIL
Research guidance (No. of Students)	Master	NIL
	PhD-	NIL
No. of papers published in	National Journals	NIL
	International Journal	NIL
	Conferences	NIL
Master (Completed /Ongoing)	ONGOING	
PhD (Completed/Ongoing)	NIL	
Projects Carried Out	NIL	
Patents(Filed & Granted)	NIL	
Technology Transfer	NIL	
Research Publications (No. of papers published in National/International/Journals/Conferences)	NIL	
No. of Books published with Details(Name of Book, Publisher with ISBN, Year of Publication etc.)	NIL	

Unit-1.0: Basics of html, php

1.1 Html form tag

1.1.1 form Attributes: action,method, target

1.1.2 form Elements: input,select, textarea, datalist, button

1.2 Web Server

1.2.1 Overview , working,List of available Web Server

1.3 Server Side Script

1.3.1 Difference between Client and Server Side Script i.e. HTMLVs PHP

1.3.2 Introduction of php, creating phpfile , features of php

HTML Form

An **HTML form** is a *section of a document* which contains controls such as text fields, password fields, checkboxes, radio buttons, submit button, menus etc.

An HTML form facilitates the user to enter data that is to be sent to the server for processing such as name, email address, password, phone number, etc. .

Why use HTML Form

HTML forms are required if you want to collect some data from of the site visitor.

For example: If a user want to purchase some items on internet, he/she must fill the form such as shipping address and credit/debit card details so that item can be sent to the given address.

HTML Form Syntax

1. `<form action="server url" method="get|post">`
2. `//input controls e.g. textfield, textarea, radiobutton, button`
3. `</form>`

HTML Form Tags

Tag	Description
<code><form></code>	It defines an HTML form to enter inputs by the used side.
<code><input></code>	It defines an input control.
<code><textarea></code>	It defines a multi-line input control.
<code><label></code>	It defines a label for an input element.
<code><fieldset></code>	It groups the related element in a form.
<code><legend></code>	It defines a caption for a <code><fieldset></code> element.
<code><select></code>	It defines a drop-down list.
<code><optgroup></code>	It defines a group of related options in a drop-down list.

<option>	It defines an option in a drop-down list.
<button>	It defines a clickable button.
<datalist>	It specifies a list of pre-defined options for input control.
<keygen>	It defines a key-pair generator field for forms.
<output>	It defines the result of a calculation.

HTML <form> element

The HTML <form> element provide a document section to take input from user. It provides various interactive controls for submitting information to web server such as text field, text area, password field, etc.

Note: The <form> element does not itself create a form but it is container to contain all required form elements, such as <input>, <label>, etc.

Syntax:

1. <form>
 2. //Form elements
 3. </form>
-

HTML <input> element

The HTML <input> element is fundamental form element. It is used to create form fields, to take input from user. We can apply different input filed to gather different information form user. Following is the example to show the simple text input.

1. <body>
2. <form>
3. Enter your name

4. <input type="text" name="username">
5. </form>
6. </body>

Output:-

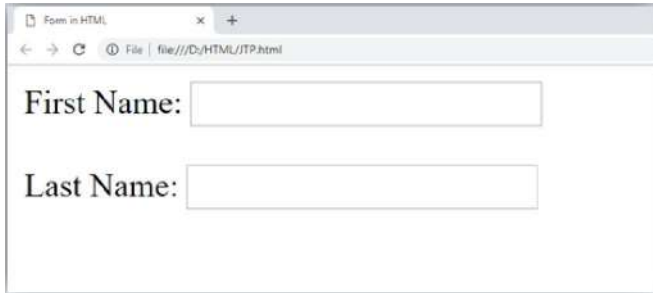
Enter your name

HTML TextField Control

The `type="text"` attribute of `input` tag creates textfield control also known as single line textfield control. The `name` attribute is optional, but it is required for the server side component such as JSP, ASP, PHP etc.

1. `<form>`
2. First Name: `<input type="text" name="firstname"/> <br/>`
3. Last Name: `<input type="text" name="lastname"/> <br/>`
4. `</form>`

Output:

A screenshot of a web browser window titled 'Form in HTML'. The address bar shows 'file:///D:/HTML/ITP.html'. The page content consists of two text input fields. The first field is preceded by the text 'First Name:' and the second field is preceded by 'Last Name:'. Both fields are empty and have a standard white background with a light gray border.

HTML `<textarea>` tag in form

The `<textarea>` tag in HTML is used to insert multiple-line text in a form. The size of `<textarea>` can be specify either using `"rows"` or `"cols"` attribute or by CSS.

Example:

1. `<!DOCTYPE html>`
2. `<html>`
3. `<head>`
4. `<title>Form in HTML</title>`
5. `</head>`
6. `<body>`
7. `<form>`
8. Enter your address:`<br>`
9. `<textarea rows="2" cols="20"></textarea>`
10. `</form>`
11. `</body>`
12. `</html>`

Output:

A screenshot of a web browser window titled 'Form in HTML'. The address bar shows 'file:///D:/HTML/ITP.html?name=Sname:'. The page content shows the text 'Enter your address:' followed by a text area. The text area is empty and has a white background with a light gray border.

The Radio Button:

The radio button is used to select one option from multiple options. It is used for selection of gender, quiz questions etc.

If you use one name for all the radio buttons, only one radio button can be selected at a time.

Using radio buttons for multiple options, you can only choose a single option at a time.

```
<form>
  <label for="gender">Gender: </label>
  <input type="radio" id="gender" name="gender" value="male"/>Male
  <input type="radio" id="gender" name="gender" value="female"/>Female <br/>
</form>
```

OUTPUT:-

Gender: ☐ Male ☒ Female

Select Tag in HTML

In HTML, with the help of a select tag, we can create the dropdown list while creating the form where the user can choose one of the options from the given option. If the developer allows the user, then the user also selects multiple options from the given option. We can implement the select tag only inside the HTML form. We can preserve the space by choosing the dropdown menu while accepting the information from the users.

We can take the selected elements inside the opening and closing of <select> tag. The <select> tag's implementation is in the example below.

Example 1:

```
1. <!DOCTYPE html>
2. <html>
3. <head>
4. <title>T-Shirt Color Selection</title>
5. <style>
6.   body {
7.     font-family: Arial, sans-serif;
8.     background-color: #f2f2f2;
9.   }
10.
11.     label {
12.       font-size: 18px;
13.       font-weight: bold;
14.       color: #333;
15.     }
16.
```

```

17.     select {
18.         font-size: 16px;
19.         padding: 8px;
20.         margin: 5px 0;
21.         border: 1px solid #ccc;
22.         border-radius: 5px;
23.         width: 200px;
24.         background-color: #fff;
25.         color: #555;
26.     }
27.
28.     option {
29.         color: #555;
30.     }
31. </style>
32. </head>
33. <body>
34.     <label for="shirt-colors">What color t-shirt would you like?</label><br>
35.
36.     <select name="shirts" id="shirt-colors">
37.         <option value="red">Red t-shirt</option>
38.         <option value="blue">Blue t-shirt</option>
39.         <option value="orange">Orange t-shirt</option>
40.     </select>
41. </body>
42. </html>

```

Output:



The screenshot shows a web form with a label "What color t-shirt would you like?". Below the label is a dropdown menu. The dropdown is open, displaying three options: "Red t-shirt", "Blue t-shirt", and "Orange t-shirt". The "Red t-shirt" option is highlighted, indicating it is the selected value.

HTML Datalist Tag

The HTML `<datalist>` tag is used to provide an auto complete feature on form element. It provides a list of predefined options to the users to select data.

The datalist tag is introduced in HTML5.

The `<datalist>` tag should be used with an `<input>` element that contains a "list" attribute. The value of "list" attribute is linked with the datalist id.

HTML datalist tag example

Let's see the simple example of HTML5 datalist tag. If you press A, it will show a list of cricketers starting with A letter.

1. <label>
2. Enter your favorite cricket player: Press any character

3. <input type="text" id="favCktPlayer" list="CktPlayers">
4. <datalist id="CktPlayers">
5. <option value="Sachin Tendulkar">
6. <option value="Brian Lara">
7. <option value="Jacques Kallis">
8. <option value="Ricky Ponting">
9. <option value="Rahul Dravid">
10. <option value="Shane Warne">
11. <option value="Rohit Sharma">
12. <option value="Donald Bradman">
13. <option value="Saurav Ganguly ">
14. <option value="AB diVilliers">
15. <option value="Mahendra Singh Dhoni">
16. <option value="Adam Gilchrist">
17. </datalist>
18. </label>

Output:

Enter your favorite cricket player: Press any character

HTML Button Tag

The <button> tag is used to create a clickable button within HTML form on your webpage. You can put content like text or image within the <button>.....</button> tag.

You should always specify the type attribute for a <button> tag. Different browsers use different default type for the button element.

HTML Button tag can be used inside and outside the form.

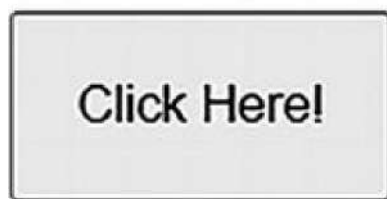
If you use it **inside the form**, it works as the submit button. You can also use it as reset button.

HTML Button Tag Example

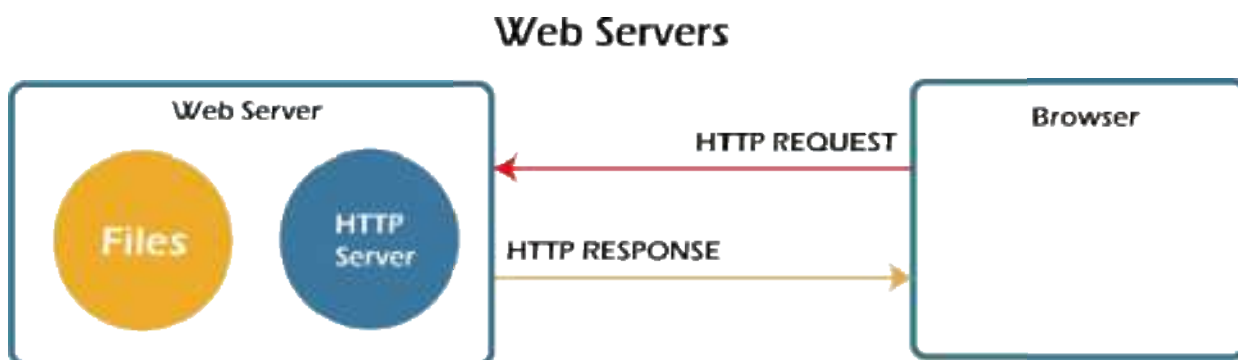
Let's see the code to display the button.

1. <button name="button" type="button">Click Here!</button>

Standard Button



Web Servers



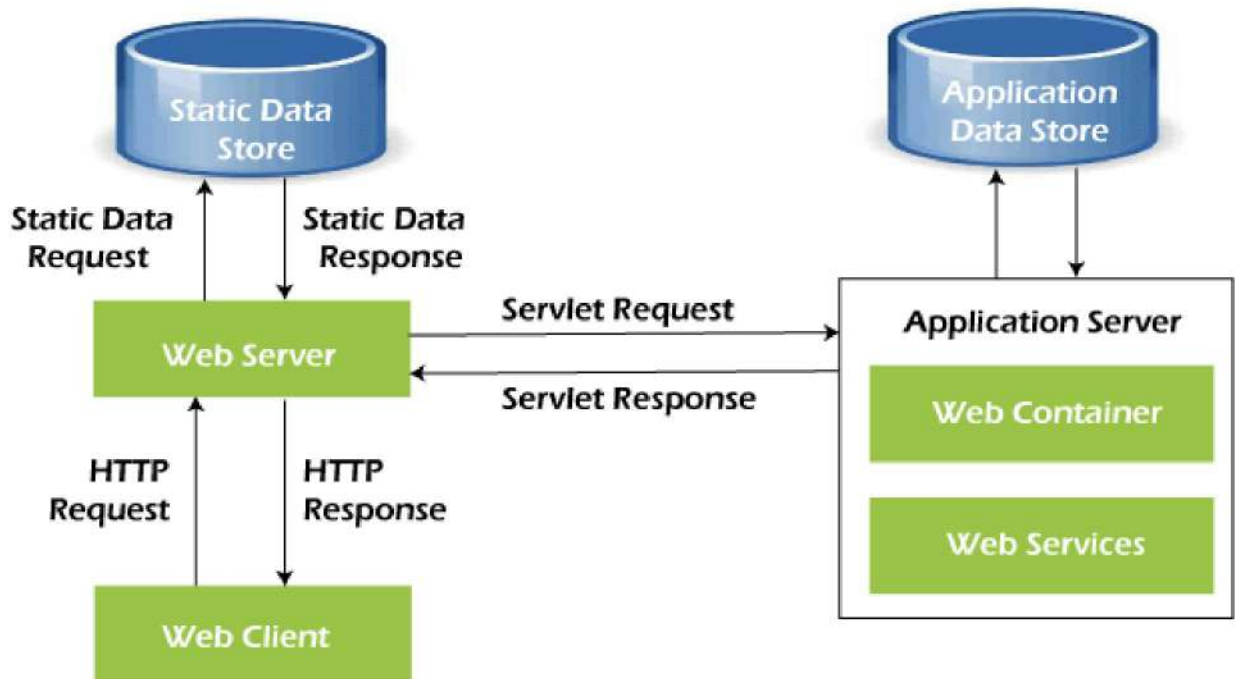
Web pages are a collection of data, including images, text files, hyperlinks, database files etc., all located on some computer (also known as server space) on the Internet. A web server is dedicated software that runs on the server-side. When any user requests their web browser to run any web page, the webserver places all the data materials together into an organized web page and forwards them back to the web browser with the help of the Internet. Therefore, we can conclude that: -

A web server is a dedicated computer responsible for running websites sitting out on those computers somewhere on the Internet. They are specialized programs that circulate web pages as summoned by the user. The primary objective of any web server is to collect, process and provide web pages to the users.

This intercommunication of a web server with a web browser is done with the help of a protocol named **HTTP (Hypertext Transfer Protocol)**. These stored web pages mostly use static content, containing **HTML** documents, images, style sheets, text files, etc. However, **web servers can serve static as well as dynamic contents**. Web Servers also assists in emailing services and storing files. Therefore it also **uses SMTP (Simple Mail Transfer Protocol)** and **FTP (File Transfer Protocol)** protocols to support the respective services. Web servers are mainly used in web hosting or hosting the website's data and running web-based applications.

The **hardware** of the web servers are connected to the Internet that manages the data exchange facility within different connected devices. In contrast, the software of web server software is responsible for controlling how a user accesses delivered files. Typically, web server management is an ideal example of the client/server model. Therefore, **it is compulsory for all computers that host websites (whether with state or dynamic web page content) to have web server software**.

Working of web servers



1. **On the hardware side**, a web server is defined as a computer that stores software and another website raw data, such as HTML files, images, text documents, and JavaScript files. The hardware of the web servers are connected to the web and supports the data exchange with different devices connected to the Internet.
2. **On the software side**, a web server includes server software accessed through website domain names. It controls how web users access the web files and ensures the supply of website content to the end-user. The web server contains several components, including an HTTP server.

Web server softwares available

Though there are various web servers found in today's market, but the commonly used one are as follows:

S.NO	Web Server Description
1.	Apache HTTP Server This web server is developed by Apache Software Foundation. It is an open-source, accessible web server available for almost all operating systems, including Windows, MACOS, Linus, FreeBSD, etc. Apache is one of the most popular web servers used around the globe.

2.	Microsoft Internet Information Services (IIS) IIS is a high-performance web server that is developed by Microsoft only for Microsoft platforms. This web server is tightly integrated with Microsoft operating system; therefore, it is not open-sourced.
3.	Nginx Nginx is an open-source web server commonly used by administrators as it supports light resource application and scalability.
4.	Lighttpd Lighttpd, also known as lighty, is a free, open-source web server with the FreeBSD operating system. This web server is fast, secure and consumes much less CPU power. It can also run on the commonly used operating system, unlike Windows, Mac OS X, Linux.
5.	Sun Java System Web Server Sun Java is a free web server developed by Sun Microsystems well equipped for a medium and large website that can run on Windows, Linux and Unix. Moreover, this web server supports several languages, scripts and technologies essential for Web 2.0, unlike JSP, Java Servlets, PHP, Python, HTML, etc. Though Sun Java is free, it is not an open-source web server.

Scripting Language

A script or scripting language is a computer language that does not need the compilation step and is rather interpreted one by one at runtime. It is where the script is written and where instructions for a run-time environment are written. In contrast to programming languages that are compiled first before running, scripting languages do not compile the file and execute the file without being compiled.

Scripts are often utilized to create dynamic web applications nowadays because they are linked to web development. Server-Side Scripting Languages and Client-Side Scripting Languages are the two types of scripting languages. Python, PHP, and Perl are examples of server-side scripting languages, while JavaScript is the greatest example of a client-side scripting language.

The scripts may be created in **two** ways: on the **client side** or the **server side**, where the server-side scripts are processed on a server. In contrast, client-side scripting needs browsers to execute scripts on the client system but doesn't connect with the server executing the client-side scripts.

In this article, you will learn about the difference between **server-side scripting** and **client-side scripting**. But before discussing the differences, you must know about server-side scripting and client-side scripting.

Server-side scripting is a programming technique for creating code that may run software on the server side. In other words, server-side scripting is any scripting method that may operate on a web server. At the server end, actions such as website customization, dynamic changes in website content, response creation to user requests, database access, and many more are carried out.

Server-side scripting creates a communication channel between a server and a client. Previously, **CGI (Common Gateway Interface)** scripts were used to implement server-side scripting, and CGI was created to execute scripts written in computer languages such as C++ or Perl on websites.

The server-side is made up of **three** parts: **the database, the server, the APIs**, and the backend web software written in the server-side scripting language. When a browser requests a page with server-side scripting, the web server evaluates the script before delivering the page to the browser. In this case, script processing may entail collecting information from a database, performing simple computations, or selecting the relevant material to be shown on the client end. The output is provided to the web browser when the script is processed. The web server hides the scripts from the end user until the content is delivered, making the data and source code safer.

Server-side scripting languages

There are various server-side scripting languages. Some main server-side scripting languages are as follows:

Python

It is an open-source language that is very powerful and easy to learn. It is suitable for beginners because it is simple to learn and read. It is believed to be used by Google and YouTube. It is a OOPs language with dynamic typing and data structures. It has grown to be one of the most popular languages for both quick application development and web development.

PHP

It is an open-source server-side scripting programming language mainly designed for web apps and is the most utilized scripting language. It allows you to retrieve and manipulate data from a database and is utilized along with SQL to query the database. It is a fast and simple language to learn and develop, and Facebook, Wikipedia, and WordPress utilize it.

Ruby

It is a free and open-source programming language that was developed and firstly introduced in the early 1990s. It is a dynamic language that is simple to read and write and an OOPs language that is interpreted as it runs. It has evolved continuously since its development and is one of the most utilized web development languages.

Client-side scripting generates code that may be executed on the client end without needing server-side processing. These scripts are typically embedded into HTML text. Client-side scripting may be utilized to check the user's form for problems before submitting it and to

change the content based on the user input. The web needs three components to function: client, database, and server.

The client-side scripting may significantly reduce server demand. It is intended to be utilized as a scripting language with a web browser as the host program. The **HTML** and **CSS** are delivered as plain text when a user uses a browser to request a webpage from the server, and the browser understands and renders the web content at the client end.

Client-side Scripting Languages

There are various client-side scripting languages. Some main client-side scripting languages are as follows:

HTML

It is not a scripting language; it is a markup language. However, it serves as the basic language for client-side web development, also referred to as front-end. The presence of hypertext on a page denotes its hyperlinks. The markup language uses tags to define the structure and layout. It is a programming language that is mainly used to design a web page's structure and layout.

CSS

CSS is an abbreviation for Cascading Style Sheets. It provides a technique for creating graphic elements that help a web application's appearance look more appealing. A style tag in a web page defines all the specifics regarding the web page's presentation, including its border styles, image styles, colour, font styles, borders, format, font size, margins, padding, etc.

JavaScript

It is a client-side scripting language designed for a specific purpose, but several JavaScript frameworks are already utilized as server-side scripting technologies.

VBScript

VBScript is based on Visual Basic, which was created by Microsoft in 1996. It is a scripting programming language that is lightweight, fast, and easy to learn. It is not a OOPs language but is similar to JavaScript.

The main differences between server-side scripting and client-side scripting are as follows:

Features	Server-side Scripting	Client-side Scripting
Primary Function	The main function of this scripting is to manipulate and grant access to the requested database.	The main purpose of this scripting is to give the requested output to the end-user.
Uses	It is employed at the backend, where the source code is invisible or concealed on the client side.	It is utilized at the front end, which users may view through the browser.
Processing	It needs server interaction.	It doesn't need any server interaction.
Security	It is more secure while working on a web app.	It is less secure than server-side scripting due to the code accessibility offered to the client.
Running	It executes on the web server.	It executes on the remote computer system.
Dependability	It doesn't depend on the client.	It depends on the user's browser version.
File Access	It offers complete access to the file that is stored in the web database server.	It doesn't offer any access to the files on the web servers.
Code Allowance	It enables the backend developer to hide the source code from the user.	The user is given access to the written code after confirming their requirements.
Occurrence	It only responds after the user begins the browsing request.	It happens when the browser processes all of the codes and then acts according to the client's needs.

Affect	It may reduce the server load.	It may effectively customize web pages and offer dynamic websites.
Languages Involved	The server-side scripting programming languages, such as PHP, ColdFusion, Python, ASP.net, Java, C++, Ruby, C#, etc.	Its programming languages are HTML, CSS, and JavaScript.

What is PHP

PHP is an open-source, interpreted, and object-oriented scripting language that can be executed at the server-side. PHP is well suited for web development. Therefore, it is used to develop web applications (an application that executes on the server and generates the dynamic page.).

PHP was created by **Rasmus Lerdorf in 1994** but appeared in the market in 1995. **PHP 7.4.0** is the latest version of PHP, which was released on **28 November**. Some important points need to be noticed about PHP are as followed:

- PHP stands for Hypertext Preprocessor.
- PHP is an interpreted language, i.e., there is no need for compilation.
- PHP is faster than other scripting languages, for example, ASP and JSP.
- PHP is a server-side scripting language, which is used to manage the dynamic content of the website.
- PHP can be embedded into HTML.
- PHP is an object-oriented language.
- PHP is an open-source scripting language.
- PHP is simple and easy to learn language.

PHP Example

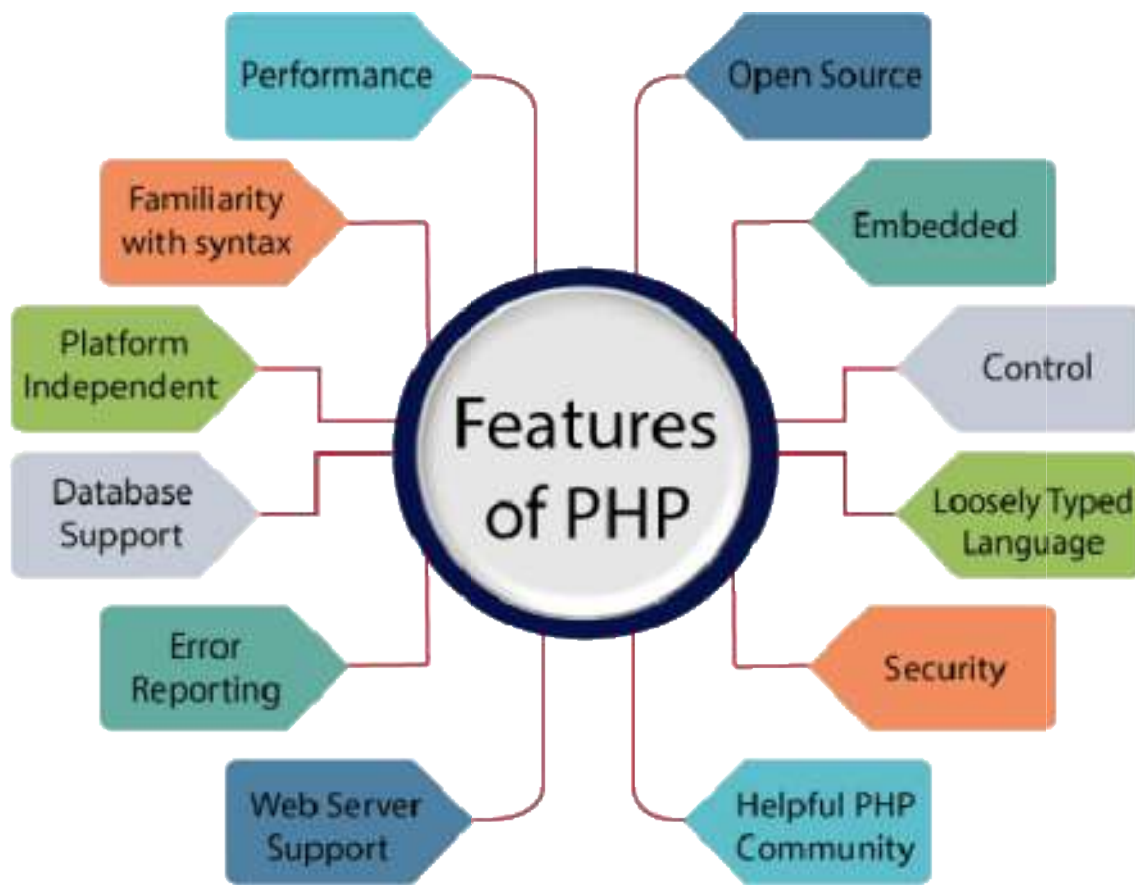
1. `<?php`
2. `echo "Hello World!";`
3. `?>`

Output:

Hello, World!

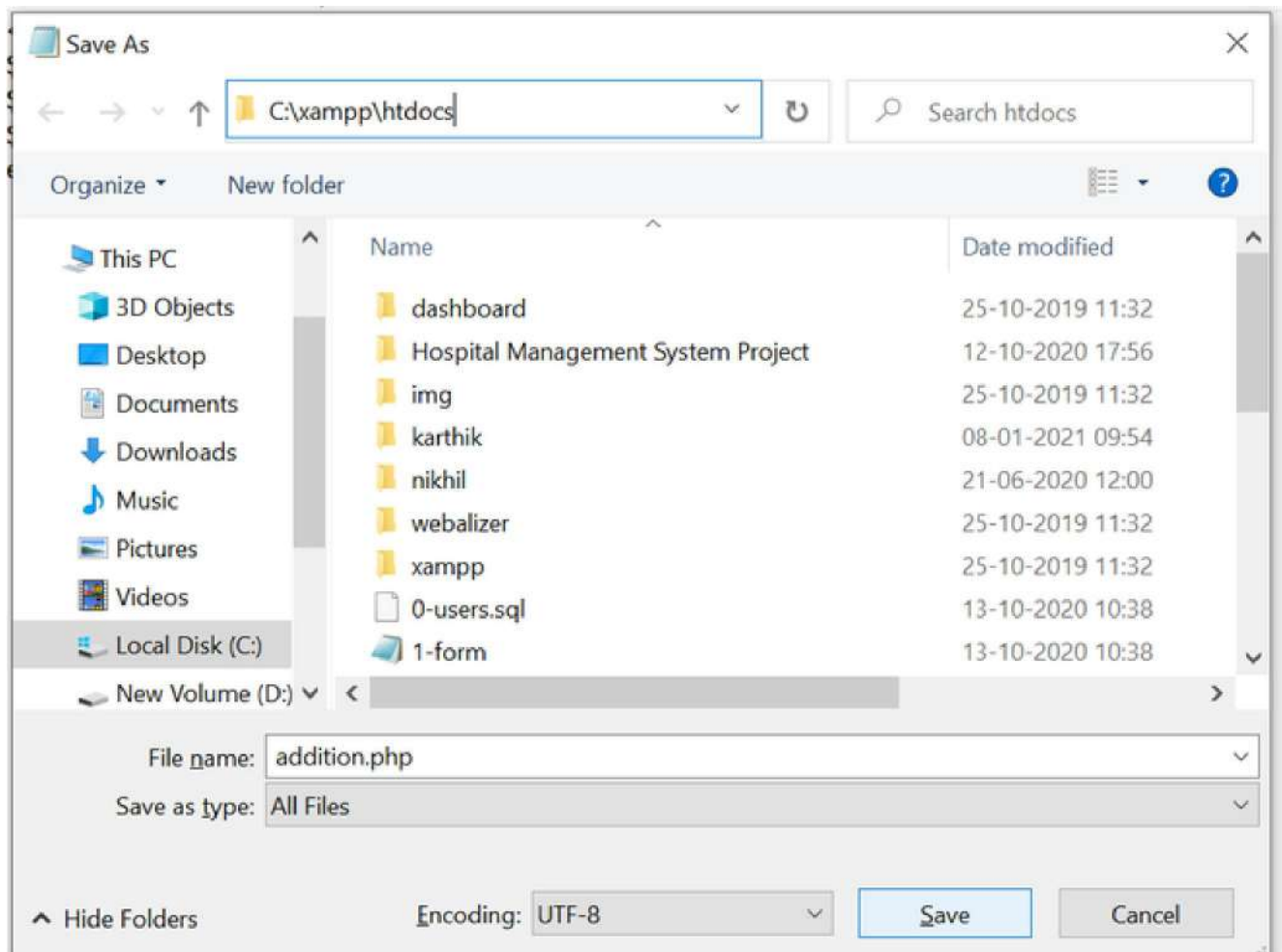
PHP Features

PHP is very popular language because of its simplicity and open source. There are some important features of PHP given below:



Ways to run a PHP program:

- Generally, we need a web browser to run a PHP program.
- Write code in notepad.
- Save the file with any name followed by a *.php* extension.
- For example, *Geeksforgeeks.php*.
- Save the file in the XAMPP location of the file structure.
- If the [XAMPP](#) is installed in local disk C, then we have to save the file in *C:\xampp\htdocs*.



Like this we have to save the PHP file

After saving your code file into the *htdocs* folder follow the below steps.

- Open XAMPP control panel.
- Start Apache and MySQL servers
- Go to any browser and type *localhost/filename.php* in the search box.
- If you save your PHP code in a subfolder in *htdocs*, then type *localhost/subfolder_name/filename.php*

A PHP code can be written in following ways:-

Without any HTML markups:

```
<?php
    $a=10;
    $b=10;
    $c=$a+$b;
    echo("The addition of a and b is ". $c);
?>
```

Output :

The addition of a and b is 20

Embedding PHP code in HTML:

- PHP

```
<html>
<body>
<?php
    echo "Your first PHP code";
?>
</body>
</html>
```

Output:

Your first PHP code

Unit-2.0: Building block of PHP

2.1 Variable, datatype, operator

2.1.1 Creating& Printing Variables

2.1.2 different datatype: integer, float,string, Boolean, array, object

2.1.3 String library function:strev(),strlen(), strword_count(), stripslashes(),strreplace(), explode(),implode()

2.1.3 Type of operator: arithmetic,logical, assignment,increment/decrement, string

2.2 Control Statements

2.2.1 if, if- else, if-elseif statements

2.2.2 Switch statement.

2.3 Loop Statements

2.3.1 while, do while statements

2.3.2 for, foreach statements

PHP Variables

In PHP, a variable is declared using a **\$ sign** followed by the variable name. Here, some important points to know about variables:

- As PHP is a loosely typed language, so we do not need to declare the data types of the variables. It automatically analyzes the values and makes conversions to its correct datatype.
- After declaring a variable, it can be reused throughout the code.
- Assignment Operator (=) is used to assign the value to a variable.

Syntax of declaring a variable in PHP is given below:

\$variablename=value;

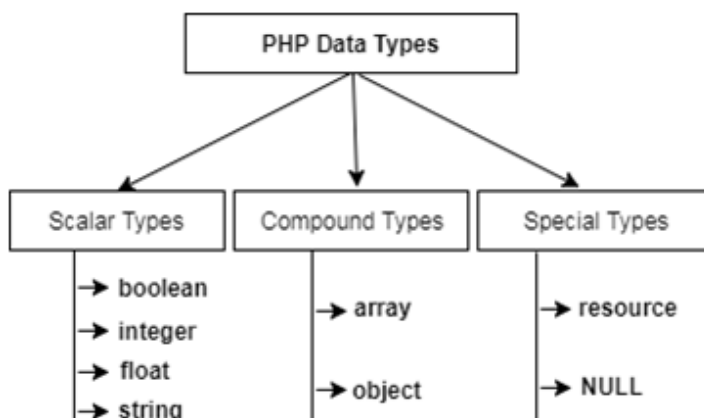
Rules for declaring PHP variable:

- A variable must start with a dollar (\$) sign, followed by the variable name.
- It can only contain alpha-numeric character and underscore (A-z, 0-9, _).
- A variable name must start with a letter or underscore (_) character.
- A PHP variable name cannot contain spaces.
- One thing to be kept in mind that the variable name cannot start with a number or special symbols.
- PHP variables are case-sensitive, so \$name and \$NAME both are treated as different variable.

PHP Data Types

PHP data types are used to hold different types of data or values. PHP supports 8 primitive data types that can be categorized further in 3 types:

1. Scalar Types (predefined)
2. Compound Types (user-defined)
3. Special Types



PHP Boolean

Booleans are the simplest data type works like switch. It holds only two values: **TRUE (1)** or **FALSE (0)**. It is often used with conditional statements. If the condition is correct, it returns TRUE otherwise FALSE.

Example:

```
<?php
if (TRUE)
echo "This condition is TRUE.";
if (FALSE)
echo "This condition is FALSE.";
?>
```

Output: This condition is TRUE

PHP Integer

Integer means numeric data with a negative or positive sign. It holds only whole numbers, i.e., numbers without fractional part or decimal points.

Rules for integer:

- An integer can be either positive or negative.
- An integer must not contain decimal point.
- Integer can be decimal (base 10), octal (base 8), or hexadecimal (base 16).
- The range of an integer must be lie between 2,147,483,648 and 2,147,483,647 i.e., -2^{31} to 2^{31} .

Example:

```
<?php
$dec1 = 34;
$oct1 = 0243;
$hexa1 = 0x45;
echo "Decimal number: " . $dec1. "</br>";
echo "Octal number: " . $oct1. "</br>";
echo "HexaDecimal number: " . $hexa1. "</br>";
?>
```

Output: *Decimal number: 34*
Octal number: 163
HexaDecimal number: 69

PHP Float

A floating-point number is a number with a decimal point. Unlike integer, it can hold numbers with a fractional or decimal point, including a negative or positive sign.

Example:

```
<?php
    $n1 = 19.34;
    $n2 = 54.472;
    $sum = $n1 + $n2;
    echo "Addition of floating numbers: " . $sum;
?>
```

Output: *Addition of floating numbers: 73.812*

PHP String

A string is a non-numeric data type. It holds letters or any alphabets, numbers, and even special characters.

String values must be enclosed either within **single quotes** or in **double quotes**. But both are treated differently. To clarify this, see the example below:

Example:

```
<?php
    $company = "Javatpoint";
    //both single and double quote statements will treat different
    echo "Hello $company";
    echo "</br>";
    echo 'Hello $company';
?>
```

Output:

Hello Javatpoint
Hello \$company

PHP Array

An array is a compound data type. It can store multiple values of same data type in a single variable.

Example:

1. `<?php`
2. `$bikes = array ("Royal Enfield", "Yamaha", "KTM");`
3. `var_dump($bikes);` //the var_dump() function returns the datatype and values
4. `echo "</br>";`
5. `echo "Array Element1: $bikes[0] </br>";`

6. `echo "Array Element2: $bikes[1] </br>";`
7. `echo "Array Element3: $bikes[2] </br>";`
8. `?>`

Output:

```
array(3) { [0]=> string(13) "Royal Enfield" [1]=> string(6) "Yamaha" [2]=> string(3) "KTM" }  
Array Element1: Royal Enfield  
Array Element2: Yamaha  
Array Element3: KTM
```

You will learn more about array in later chapters of this tutorial.

PHP object

Objects are the instances of user-defined classes that can store both values and functions. They must be explicitly declared.

Example:

```
<?php  
    class bike {  
        function model() {  
            $model_name = "Royal Enfield";  
            echo "Bike Model: " . $model_name;  
        }  
    }  
    $obj = new bike();  
    $obj -> model();  
?>
```

Output:

```
Bike Model: Royal Enfield
```

This is an advanced topic of PHP, which we will discuss later in detail.

PHP Resource

Resources are not the exact data type in PHP. Basically, these are used to store some function calls or references to external PHP resources. **For example** - a database call. It is an external resource.

This is an advanced topic of PHP, so we will discuss it later in detail with examples.

PHP Null

Null is a special data type that has only one value: **NULL**. There is a convention of writing it in capital letters as it is case sensitive.

The special type of data type NULL defined a variable with no value.

Example:

```
<?php
    $nl = NULL;
    echo $nl; //it will not give any output
?>
```

Output:

PHP Operators

PHP Operator is a symbol i.e used to perform operations on operands. In simple words, operators are used to perform operations on variables or values. For example:

\$num=10+20;//+ is the operator and 10,20 are operands

In the above example, + is the binary + operator, 10 and 20 are operands and \$num is variable.

PHP Operators can be categorized in following forms:

- [Arithmetic Operators](#)
- [Assignment Operators](#)
- [Comparison Operators](#)
- [Incrementing/Decrementing Operators](#)
- [Logical Operators](#)
- [String Operators](#)
- [Array Operators](#)

We can also categorize operators on behalf of operands. They can be categorized in 3 forms:

Advertisement

- **Unary Operators:** works on single operands such as ++, -- etc.
- **Binary Operators:** works on two operands such as binary +, -, *, / etc.
- **Ternary Operators:** works on three operands such as "?:".

Arithmetic Operators

The PHP arithmetic operators are used to perform common arithmetic operations such as addition, subtraction, etc. with numeric values.

Operator	Name	Example	Explanation
----------	------	---------	-------------

+	Addition	$\$a + \b	Sum of operands
-	Subtraction	$\$a - \b	Difference of operands
*	Multiplication	$\$a * \b	Product of operands
/	Division	$\$a / \b	Quotient of operands
%	Modulus	$\$a \% \b	Remainder of operands
**	Exponentiation	$\$a ** \b	$\$a$ raised to the power $\$b$

The exponentiation (**) operator has been introduced in PHP 5.6.

Assignment Operators

The assignment operators are used to assign value to different variables. The basic assignment operator is "=".

Operator	Name	Example	Explanation
=	Assign	$\$a = \b	The value of right operand is assigned to the left operand.
+=	Add then Assign	$\$a += \b	Addition same as $\$a = \$a + \$b$

<code>-=</code>	Subtract then Assign	<code>\$a -= \$b</code>	Subtraction same as <code>\$a = \$a - \$b</code>
<code>*=</code>	Multiply then Assign	<code>\$a *= \$b</code>	Multiplication same as <code>\$a = \$a * \$b</code>
<code>/=</code>	Divide then Assign (quotient)	<code>\$a /= \$b</code>	Find quotient same as <code>\$a = \$a / \$b</code>
<code>%=</code>	Divide then Assign (remainder)	<code>\$a %= \$b</code>	Find remainder same as <code>\$a = \$a % \$b</code>

Incrementing/Decrementing Operators

The increment and decrement operators are used to increase and decrease the value of a variable.

Operator	Name	Example	Explanation
<code>++</code>	Increment	<code>++\$a</code>	Increment the value of <code>\$a</code> by one, then return <code>\$a</code>
		<code>\$a++</code>	Return <code>\$a</code> , then increment the value of <code>\$a</code> by one
<code>--</code>	decrement	<code>--\$a</code>	Decrement the value of <code>\$a</code> by one, then return <code>\$a</code>

		\$a--	Return \$a, then decrement the value of \$a by one
--	--	-------	--

Logical Operators

The logical operators are used to perform bit-level operations on operands. These operators allow the evaluation and manipulation of specific bits within the integer.

Operator	Name	Example	Explanation
and	And	\$a and \$b	Return TRUE if both \$a and \$b are true
Or	Or	\$a or \$b	Return TRUE if either \$a or \$b is true
xor	Xor	\$a xor \$b	Return TRUE if either \$ or \$b is true but not both
!	Not	! \$a	Return TRUE if \$a is not true
&&	And	\$a && \$b	Return TRUE if either \$a and \$b are true

	Or	\$a \$b	Return TRUE if either \$a or \$b is true
--	----	------------	--

String Operators

The string operators are used to perform the operation on strings. There are two string operators in PHP, which are given below:

Operator	Name	Example	Explanation
.	Concatenation	\$a . \$b	Concatenate both \$a and \$b
.=	Concatenation and Assignment	\$a .= \$b	First concatenate \$a and \$b, then assign the concatenated string to \$a, e.g. \$a = \$a . \$b

PHP Variable: Declaring string, integer, and float

Let's see the example to store string, integer, and float values in PHP variables.

File: variable1.php

1. `<?php`
2. `$str="hello string";`
3. `$x=200;`
4. `$y=44.6;`
5. `echo "string is: $str <br/>";`
6. `echo "integer is: $x <br/>";`
7. `echo "float is: $y <br/>";`
8. `?>`

Output:

string is: hello string
integer is: 200
float is: 44.6

PHP Variable: Sum of two variables

File: variable2.php

1. `<?php`
2. `$x=5;`
3. `$y=6;`
4. `$z=$x+$y;`
5. `echo $z;`
6. `?>`

Output:

11

PHP Variable: case sensitive

In PHP, variable names are case sensitive. So variable name "color" is different from Color, COLOR, COLor etc.

strrev() :-

The strrev() function is predefined function of PHP. It is used to reverse a string. It is one of the most basic string operations which are used by programmers and developers.

Syntax:

string strrev (string \$string);

Example 1

1. `<?php`
2. `$var1="Hello PHP";`
3. `echo "Your Number is: ".$var1;`
4. `echo "<br>";`
5. `echo "By using 'strrev()' function: ".strrev("$var1");`
6. `?>`

Output:

*Your Number is:Hello PHP
By using 'strrev()' function:PHP olleH*

strlen():-

The strlen() function is predefined function of PHP. It is used to find the length of string or returns the length of a string including all the whitespaces and special character.

Syntax:

1. strlen(string);

Example 1

1. <?php
2. \$str = 'Javatpoint';
3. echo "Your String is:". \$str;
4. echo "
";
5. echo "By using 'strlen()' function:".strlen(\$str); // 10
6. ?>

Output:

*Your String is:Javatpoint
By using 'strlen()' function:10*

str_word_count():-

The str_word_count() is in-built function of PHP. It is used to return information about words used in a string or counts the number of words in a string.

Syntax:

1. str_word_count(string,**return**,char)

Example 1

1. <?php
2. \$str="PHP Javatpoint";
3. echo "Your string is:". \$str;
4. echo "
";
5. echo "By using str_word_count(): ".str_word_count(\$str);
6. ?>

Output:

*Your string is:PHP Javatpoint
By using str_word_count(): 2*

strpos():

The strpos() is in-built function of PHP. It is used to find the position of the first occurrence of a string inside another string or substring in a string.

Syntax:

1. int strpos (string \$haystack , mixed \$needle [, int \$offset = 0]);

Example 1

```
1. <?php
2. $str1="Hello Php";
3. $str2="Hello Php javatpoint!";
4. echo "First string is: ".$str1;
5. echo "<br>";
6. echo "First string is: ".$str2;
7. echo "<br>";
8. echo "By using 'strpos()' function:".strpos("$str1,$str2","php");
9. // $str1 and $str2 'Php' first letter is upper case so output is nothing.
10. // It is case-sensitive
11. ?>
```

Output:

*First string is: Hello Php
First string is: Hello Php javatpoint!
By using 'strpos()' function:*

Example 2

```
1. <?php
2. $str1="Hello php";
3. $str2=" Hello php javatpoint!";
4. echo "First string is: ".$str1;
5. echo "<br>";
6. echo "First string is: ".$str2;
7. echo "<br>";
8. echo "By using 'strpos()' function:".strpos("$str1,$str2","php");
9. // Find the position of the first occurrence of "php" inside the string
10. ?>
```

Output:

```
First string is: Hello php
First string is: Hello php javatpoint!
By using 'strpos()' function:6
```

str_replace() :

The str_replace() function is a **case-sensitive**, built-in function of PHP which replaces some character of the string with other characters. It is used to replace all the occurrences of the search string with the replacement string.

Syntax

The syntax of the str_replace() function is given below, which has the following four parameters.

1. str_replace (\$search, \$replace, \$string, \$count)

Example 1: Basic example with string variable

1. <?php
2. \$string = "Hii everyone!";
3. \$search = 'Hii';
4. \$replace = 'Hello';
5. echo ''. "String before replacement:". "</br>';
6. echo \$string. "</br>';
7. \$newstr = str_replace(\$search, \$replace, \$string, \$count);
8. echo ''. "New replaced string is:". "</br>';
9. echo \$newstr. "</br>';
10. echo 'Number of replacement = '.\$count;
11. ?>

Output:

In the above example, we can see that "Hii" is replaced with "Hello" and the number of replacement is only 1.

String before replacement:

Hii everyone!

New replaced string is:

Hello everyone!

Number of replacement = 1

explode():-

PHP explode() is a string function, which splits a string by a string. In simple words, we can say that it breaks a string into an array. The explode() function has a "**separator**" parameter, which cannot contain an empty string, because it holds the original string that is to be split. It is a binary-safe function.

The explode() function returns an array of strings created by splitting the original [string](#).

Syntax:

1. explode (string \$separator, string \$originalString, **int** \$limit)

Parameters

There are three parameters passed in the explode() function, in which two parameters are mandatory, and the last one is optional to pass. These parameters are as follows:

\$separator:

This parameter specifies the character for the point at which the original string will split. In simple words, we can say that whenever this character is found in the string, the string will be divided into parts.

\$originalString:

This parameter holds the string which is to be split into array.

\$limit:

The \$limit parameter specifies the number of array elements to be returned. It can contain any integer value (zero, positive, or negative).

Possible values for \$limit:

Positive (Greater than 0)	If this parameter contains a positive value, this function returns an array of string, split into a size defined in the \$limit parameter.
Negative (Less than 0)	If the \$limit parameter consists of the negative value, elements from the last will be trimmed, and the remaining will be returned.
Zero	If zero (0) is passed in \$limit parameter, it will return the whole string as a single array element.

Note: Remember that, if the \$limit parameter is not provided in explode() function, the returned array will contain all elements of string separated by the \$separator string.

Return Value

This function returns an **array of strings**. This array of string is formed by splitting the original string.

Example 1: Array with \$limit parameter

1. <?php


```

2. // original string
3. $Original_str = "Hello, we are here to help you.";
4.
5. // Passed zero
6. print_r (explode (" ", $Original_str, 0));
7. // Passed positive value
8. print_r (explode (" ", $Original_str, 4));
9. // Passed negative value
10. print_r (explode (" ", $Original_str, -3));
11. ?>

```

Output:

In the above example, a space character is used as separator to split the string.

```

Array ( [0] => Hello, we are here to help you. ) Array ( [0] => Hello, [1] => we [2] => are [3] => here to help you. )
Array ( [0] => Hello, [1] => we [2] => are [3] => here )

```

The above output can be seen as to better understand:

```

Array (
[0] => Hello, we are here to help you.
) Array (
[0] => Hello,
[1] => we
[2] => are
[3] => here to help you.
) Array (
[0] => Hello,
[1] => we
[2] => are
[3] => here
)

```

Example 1: Array without \$limit parameter

```

1. <?php
2. // original string
3. $Original_str = "Hello, welcome to javatpoint.";
4. //without passing optional parameter
5. print_r (explode (" ", $Original_str));
6. ?>

```

Output:

In the above code, we do not pass the optional parameter, i.e., \$limit. Therefore, the explode() function splits the string into array for different indexes.

```

Array ( [0] => Hello, [1] => welcome [2] => to [3] => javatpoint. )

```

Example 3:

```

1. <?php

```

```
2. // original string
3. $Original_str = "Hello, welcome to javatpoint.";
4. //without passing optional parameter
5. print_r (explode ("e", $Original_str));
6. ?>
```

Output:

In the above code, we used the "e" character to split the string into an array. So, wherever the "e" is found, the string will be split.

```
Array ( [0] => H [1] => llo, w [2] => lcom [3] => to javatpoint. )
```

implode() :

PHP implode() is a string function, which joins the array elements in a string. It is a **binary-safe** function. In implode() function, parameters can be passed in any order.

The implode() function works same as the join() function and returns a string created from the elements of the array. Basically, this function joins all elements of array in one string.

Syntax

There are two syntax are available for implode() function, which are given below:

```
implode (string $glue, array $pieces)
or
implode (array $pieces)
```

Parameters

There are two parameters can be passed in the implode() function, one of which is mandatory, and another one is optional. These parameters are as follows:

\$glue (optional):

It is an optional and string type of parameter. It contains the value to join the array elements and form a string. Basically, \$glue is used to join the string.

\$pieces (mandatory):

This parameter contains the array of string to implode. The array elements are mandatory to pass in implode() function to join into one string.

Return Value

The implode() function returns the string formed from the array elements. The string will be formed in the same order as elements passed into array. The return type of this function is string.

Example 1:

In the below example, array elements are joined with + operator using implode() function.

```
1. <?php
2.     echo "Before using 'implode()' function: <br>";
3.     echo "array('Welcome', 'to', 'PHP', 'tutorial') <br> <br>";
4.
5.     //store array element in a variable
6.     $arr = array('Welcome', 'to', 'PHP', 'tutorial');
7.
8.     //join array elements in a string by + operator
9.     echo "After using 'implode()' function: <br>";
10.    echo implode("+",$arr);
11.    ?>
```

Output:

Before using 'implode()' function:
array('Welcome', 'to', 'PHP', 'tutorial')

After using 'implode()' function:
Welcome+to+PHP+tutorial

PHP If Else

PHP if else statement is used to test condition. There are various ways to use if statement in PHP.

- [if](#)
- [if-else](#)
- [if-else-if](#)
- [nested if](#)

PHP If Statement

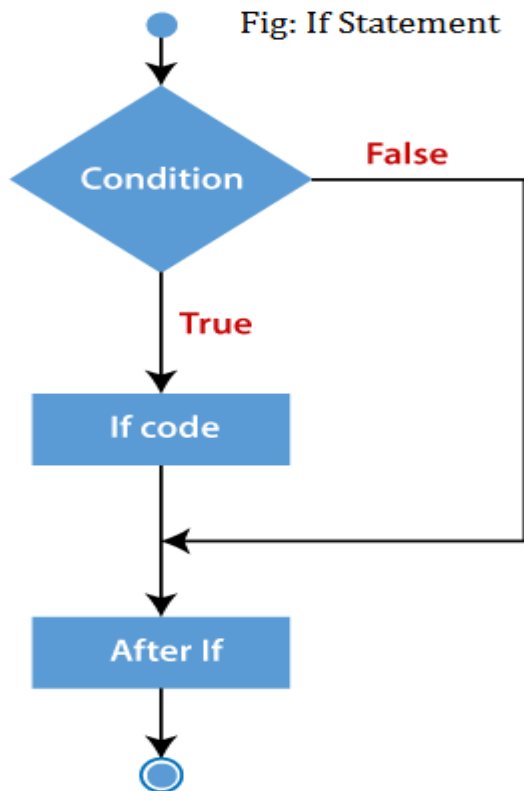
PHP if statement allows conditional execution of code. It is executed if condition is true.

If statement is used to executes the block of code exist inside the if statement only if the specified condition is true.

Syntax

```
1. if(condition){
2. //code to be executed
3. }
```

Flowchart



Example

```
1. <?php
2. $num=12;
3. if($num<100){
4. echo "$num is less than 100";
5. }
6. ?>
```

Output:

12 is less than 100

PHP If-else Statement

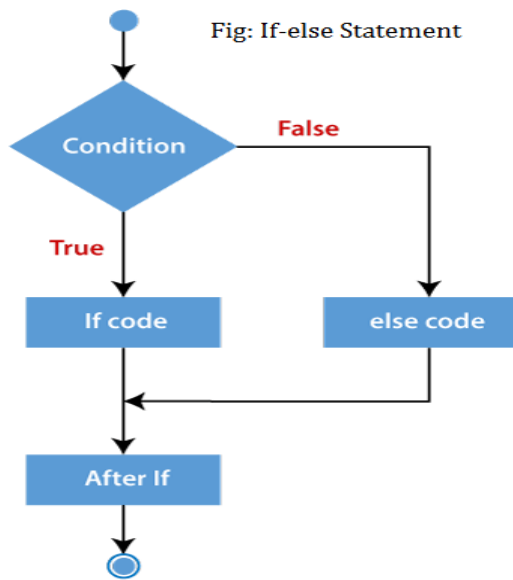
PHP if-else statement is executed whether condition is true or false.

If-else statement is slightly different from if statement. It executes one block of code if the specified condition is **true** and another block of code if the condition is **false**.

Syntax

```
1. if(condition){
2. //code to be executed if true
3. }else{
4. //code to be executed if false
5. }
```

Flowchart



```

1. <?php
2. $num=12;
3. if($num%2==0){
4. echo "$num is even number";
5. }else{
6. echo "$num is odd number";
7. }
8. ?>
  
```

Output:

12 is even number

PHP If-else-if Statement

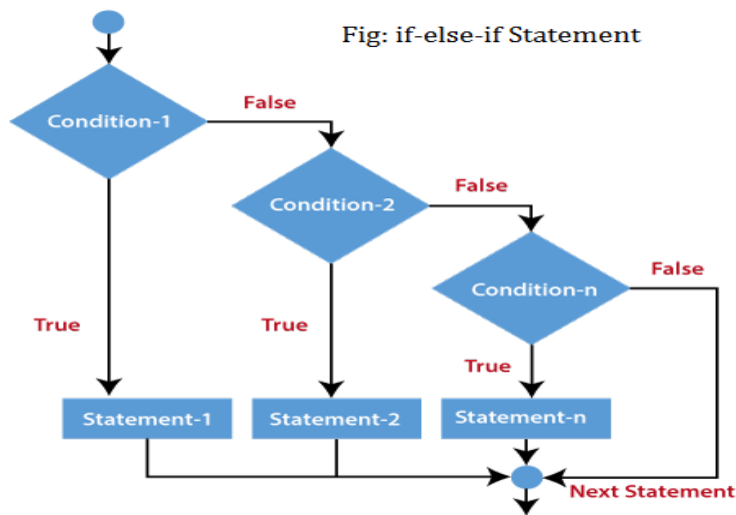
The PHP if-else-if is a special statement used to combine multiple if-else statements. So, we can check multiple conditions using this statement.

Syntax

```

1. if (condition1){
2. //code to be executed if condition1 is true
3. } elseif (condition2){
4. //code to be executed if condition2 is true
5. } elseif (condition3){
6. //code to be executed if condition3 is true
7. ....
8. } else{
9. //code to be executed if all given conditions are false
10. }
  
```

Flowchart



Example

```

1. <?php
2.   $marks=69;
3.   if ($marks<33){
4.     echo "fail";
5.   }
6.   else if ($marks>=34 && $marks<50) {
7.     echo "D grade";
8.   }
9.   else if ($marks>=50 && $marks<65) {
10.    echo "C grade";
11.  }
12.    else if ($marks>=65 && $marks<80) {
13.      echo "B grade";
14.    }
15.    else if ($marks>=80 && $marks<90) {
16.      echo "A grade";
17.    }
18.    else if ($marks>=90 && $marks<100) {
19.      echo "A+ grade";
20.    }
21.    else {
22.      echo "Invalid input";
23.    }
24.  ?>
  
```

Output:

B Grade

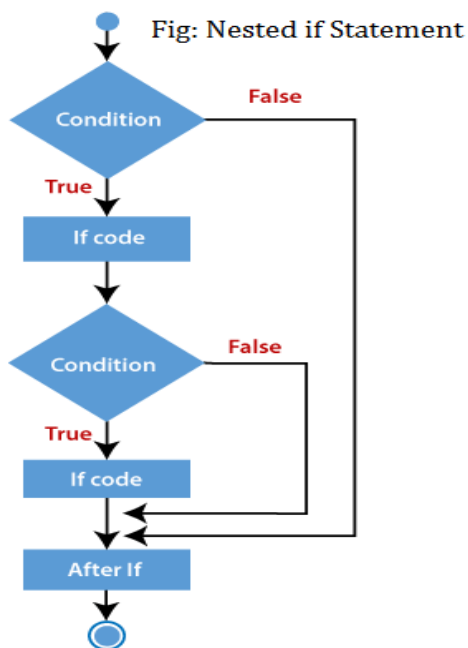
PHP nested if Statement

The nested if statement contains the if block inside another if block. The inner if statement executes only when specified condition in outer if statement is **true**.

Syntax

1. **if** (condition) {
2. //code to be executed if condition is true
3. **if** (condition) {
4. //code to be executed if condition is true
5. }
6. }

Flowchart



Example

```
1. <?php
2.     $age = 23;
3.     $nationality = "Indian";
4.     //applying conditions on nationality and age
5.     if ($nationality == "Indian")
6.     {
7.         if ($age >= 18) {
8.             echo "Eligible to give vote";
9.         }
10.        else {
11.            echo "Not eligible to give vote";
12.        }
13.    }
14.    ?>
```

Output:

Eligible to give vote

PHP Switch Example

```
1. <?php
2.     $a = 34; $b = 56; $c = 45;
3.     if ($a < $b) {
4.         if ($a < $c) {
5.             echo "$a is smaller than $b and $c";
6.         }
7.     }
8. ?>
```

Output:

34 is smaller than 56 and 45

PHP Switch

PHP switch statement is used to execute one statement from multiple conditions. It works like PHP if-else-if statement.

Syntax

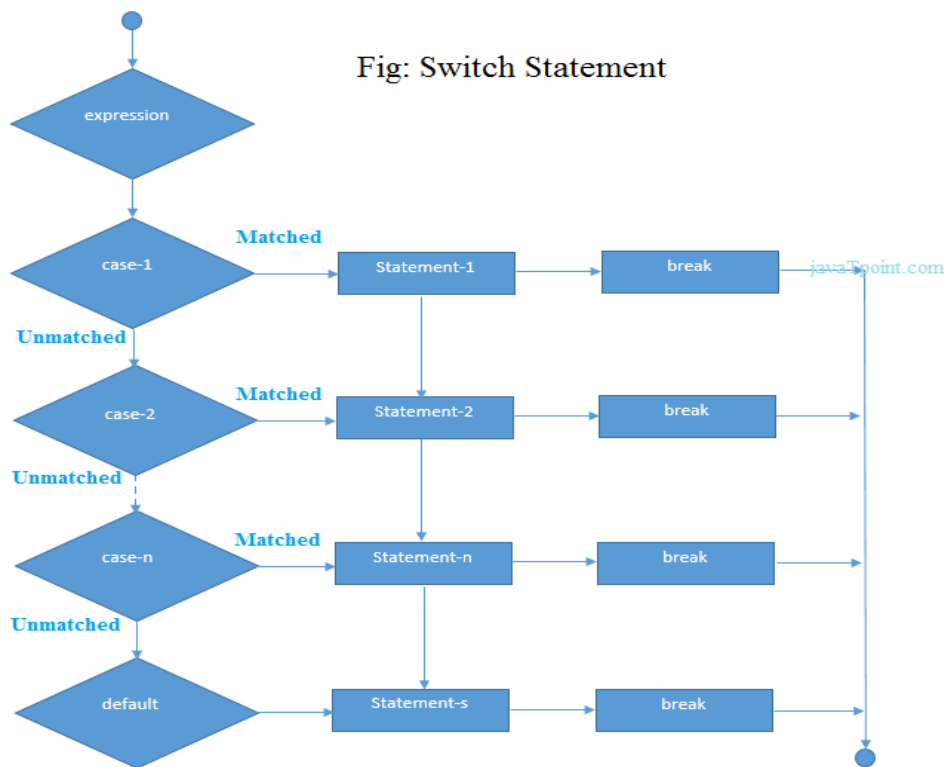
```
1. switch(expression){
2. case value1:
3.     //code to be executed
4.     break;
5. case value2:
6.     //code to be executed
7.     break;
8. ....
9. default:
10.    code to be executed if all cases are not matched;
11. }
```

Important points to be noticed about switch case:

1. The **default** is an optional statement. Even it is not important, that default must always be the last statement.
2. There can be only one **default** in a switch statement. More than one default may lead to a **Fatal** error.
3. Each case can have a **break** statement, which is used to terminate the sequence of statement.
4. The **break** statement is optional to use in switch. If break is not used, all the statements will execute after finding matched case value.
5. PHP allows you to use number, character, string, as well as functions in switch expression.
6. Nesting of switch statements is allowed, but it makes the program more complex and less readable.

7. You can use semicolon (;) instead of colon (:). It will not generate any error.

PHP Switch Flowchart



PHP Switch Example

```
<?php
$num=20;
switch($num){
case 10:
echo("number is equals to 10");
break;
case 20:
echo("number is equal to 20");
break;
case 30:
echo("number is equal to 30");
break;
default:
echo("number is not equal to 10, 20 or 30");
}
?>
```

Output:

number is equal to 20

PHP For Loop

PHP for loop can be used to traverse set of code for the specified number of times.

It should be used if the number of iterations is known otherwise use while loop. This means for loop is used when you already know how many times you want to execute a block of code.

It allows users to put all the loop related statements in one place. See in the syntax given below:

Syntax

1. **for**(initialization; condition; increment/decrement){
2. //code to be executed
3. }

Parameters

The php for loop is similar to the java/C/C++ for loop. The parameters of for loop have the following meanings:

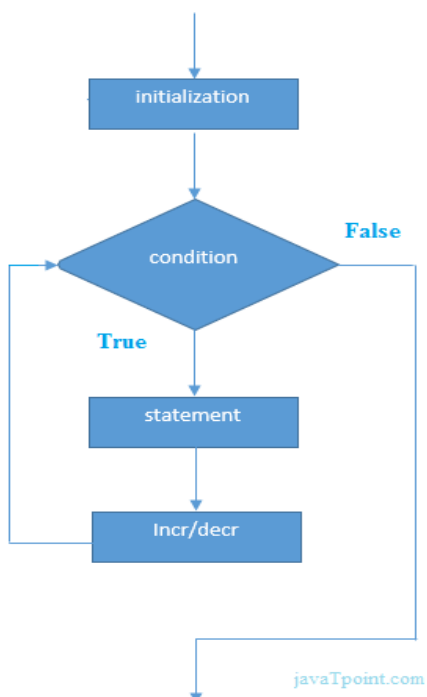
Advertisement

initialization - Initialize the loop counter value. The initial value of the for loop is done only once. This parameter is optional.

condition - Evaluate each iteration value. The loop continuously executes until the condition is false. If TRUE, the loop execution continues, otherwise the execution of the loop ends.

Increment/decrement - It increments or decrements the value of the variable.

Flowchart



Example

1. <?php
2. **for**(\$n=1;\$n<=10;\$n++){
3. echo "\$n
";
4. }
5. ?>

Output:

```
1
2
3
4
5
6
7
8
9
10
```

PHP Nested For Loop

We can use for loop inside for loop in PHP, it is known as nested for loop. The inner for loop executes only when the outer for loop condition is found **true**.

In case of inner or nested for loop, nested for loop is executed fully for one outer for loop. If outer for loop is to be executed for 3 times and inner for loop for 3 times, inner for loop will be executed 9 times (3 times for 1st outer loop, 3 times for 2nd outer loop and 3 times for 3rd outer loop).

Example

1. <?php
2. **for**(\$i=1;\$i<=3;\$i++){
3. **for**(\$j=1;\$j<=3;\$j++){
4. echo "\$i \$j
";
5. }
6. }
7. ?>

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

PHP For Each Loop

PHP for each loop is used to traverse array elements.

Syntax

1. **foreach**(\$array **as** \$var){
2. //code to be executed
3. }
4. ?>

Example

1. <?php
2. \$season=**array**("summer","winter","spring","autumn");
3. **foreach**(\$season **as** \$arr){
4. echo "Season is: \$arr
";
5. }
6. ?>

Output:

```
Season is: summer
Season is: winter
Season is: spring
Season is: autumn
```

PHP While Loop

PHP while loop can be used to traverse set of code like for loop. The while loop executes a block of code repeatedly until the condition is FALSE. Once the condition gets FALSE, it exits from the body of loop.

It should be used if the number of iterations is not known.

The while loop is also called an **Entry control loop** because the condition is checked before entering the loop body. This means that first the condition is checked. If the condition is true, the block of code will be executed.

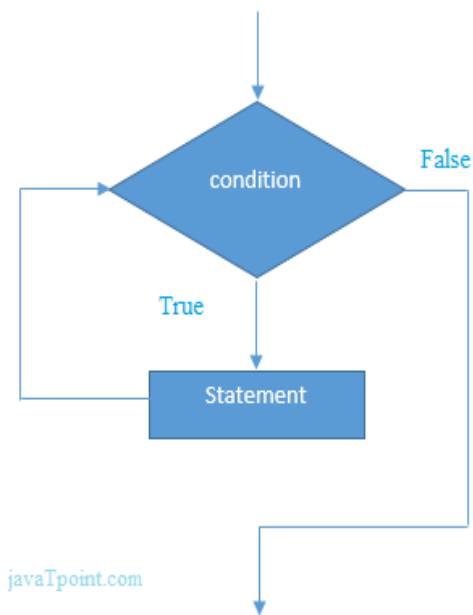
Syntax

1. **while**(condition){
2. //code to be executed
3. }

Alternative Syntax

1. **while**(condition):
2. //code to be executed
- 3.
4. **endwhile**;

PHP While Loop Flowchart



PHP While Loop Example

1. <?php
2. \$n=1;
3. **while**(\$n<=10){
4. echo "\$n
";
5. \$n++;
6. }
7. ?>

Output:

```
1
2
3
4
5
6
7
```

```
8  
9  
10
```

Alternative Example

1. <?php
2. \$n=1;
3. **while**(\$n<=10):
4. echo "\$n
";
5. \$n++;
6. **endwhile**;
7. ?>

Output:

```
1  
2  
3  
4  
5  
6  
7  
8  
9
```

PHP do-while loop

PHP do-while loop can be used to traverse set of code like php while loop. The PHP do-while loop is guaranteed to run at least once.

The PHP do-while loop is used to execute a set of code of the program several times. If you have to execute the loop at least once and the number of iterations is not even fixed, it is recommended to use the **do-while** loop.

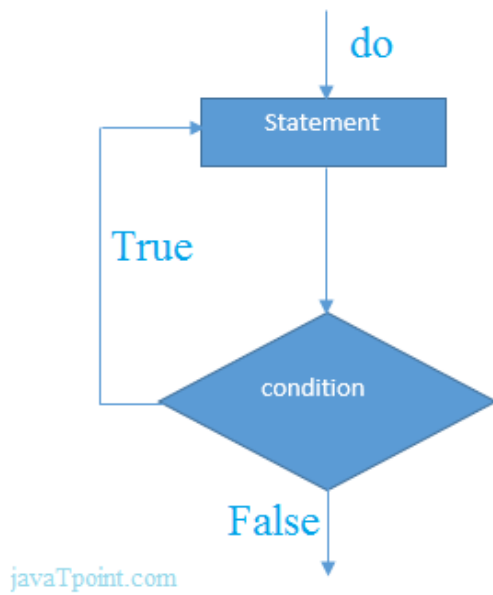
It executes the code at least one time always because the condition is checked after executing the code.

The do-while loop is very much similar to the while loop except the condition check. The main difference between both loops is that while loop checks the condition at the beginning, whereas do-while loop checks the condition at the end of the loop.

Syntax

1. **do**{
2. //code to be executed
3. }**while**(condition);

Flowchart



Example

1. <?php
2. \$n=1;
3. **do**{
4. echo "\$n
";
5. \$n++;
6. }**while**(\$n<=10);
7. ?>

Output:

```
1
2
3
4
5
6
7
8
9
10
```

Example

A semicolon is used to terminate the do-while loop. If you don't use a semicolon after the do-while loop, it is must that the program should not contain any other statements after the do-while loop. In this case, it will not generate any error.

1. <?php
2. \$x = 5;
3. **do** {

```
4.     echo "Welcome to javatpoint! </br>";
5.     $x++;
6. } while ($x < 10);
7. ?>
```

Output:

```
Welcome to javatpoint!
Welcome to javatpoint!
Welcome to javatpoint!
Welcome to javatpoint!
Welcome to javatpoint!
```

User input in PHP

Readline():-

The **readline()** function is an inbuilt function in PHP that is used to read user input from the command line window. It provides a way to interactively prompt the user for input and capture their response as a string.

Syntax:

```
readline($prompt): string|false
```

Parameters: This function accepts one parameter which is described below.

- **\$prompt:** This is an optional parameter. This is the string parameter that is displayed when the user entered the command line interface.

Return Value: The **readline()** function returns the provided string. This function will return false if no more data is found to be read.

Program 1: The following program demonstrates the readline() function.

```
<?php
// Simple example prompting the user for their name
$name = readline("Enter your name: ");

// Check if the user provided a name
if ($name !== "") {
    echo "Hello, $name! Nice to meet you.";
} else {
    echo "You didn't enter your name.";
}
?>
```

Output

```
Enter your name: Hello, ! Nice to meet you.
```

Unit-3.0: Function and Data

Handling

3.1 creating user defined function

3.1.1 without arguments

3.1.2 arguments with default value

3.1.3 function returning value

3.2 header()function

3.2.1 http redirect

3.2.2 content - disposition

3.3 Data Handling

3.3.1 superglobals in php :\$globals,
\$_server, \$_request, \$_post,
\$_get, \$_cookie, \$_session
3.3.2 need of superglobals

PHP Functions

PHP function is a piece of code that can be reused many times. It can take input as argument list and return value. There are thousands of built-in functions in PHP.

Advantage of PHP Functions

Code Reusability: PHP functions are defined only once and can be invoked many times, like in other programming languages.

Less Code: It saves a lot of code because you don't need to write the logic many times. By the use of function, you can write the logic only once and reuse it.

Easy to understand: PHP functions separate the programming logic. So it is easier to understand the flow of the application because every logic is divided in the form of functions.

PHP User-defined Functions

We can declare and call user-defined functions easily. Let's see the syntax to declare user-defined functions.

Syntax

1. **function** functionname(){
2. //code to be executed
3. }

PHP Functions Example

File: function1.php

1. <?php
2. **function** sayHello(){
3. echo "Hello PHP Function";
4. }
5. sayHello();//calling function
6. ?>

Output:

Hello PHP Function

PHP Function Arguments

We can pass the information in PHP function through arguments which is separated by comma.

PHP supports **Call by Value** (default), **Call by Reference**, **Default argument values** and **Variable-length argument list**.

Let's see the example to pass single argument in PHP function.

File: functionarg.php

1. <?php
2. **function** sayHello(\$name){
3. echo "Hello \$name
";
4. }
5. sayHello("Sonoo");
6. sayHello("Vimal");
7. sayHello("John");
8. ?>

Output:

Hello Sonoo

*Hello Vimal
Hello John*

File: functionarg2.php

```
1. <?php
2. function sayHello($name,$age){
3. echo "Hello $name, you are $age years old<br/>";
4. }
5. sayHello("Sonoo",27);
6. sayHello("Vimal",29);
7. sayHello("John",23);
8. ?>
```

Output:

*Hello Sonoo, you are 27 years old
Hello Vimal, you are 29 years old
Hello John, you are 23 years old*

PHP Call By Reference

Value passed to the function doesn't modify the actual value by default (call by value). But we can do so by passing value as a reference.

By default, value passed to the function is call by value. To pass value as a reference, you need to use ampersand (&) symbol before the argument name.

Let's see a simple example of call by reference in PHP.

File: functionref.php

```
1. <?php
2. function adder(&$str2)
3. {
4.     $str2 .= 'Call By Reference';
5. }
6. $str = 'Hello ';
7. adder($str);
8. echo $str;
9. ?>
```

Output:

Hello Call By Reference

PHP Function: Default Argument Value

We can specify a default argument value in function. While calling PHP function if you don't specify any argument, it will take the default argument. Let's see a simple example of using default argument value in PHP function.

File: functiondefaultarg.php

```
1. <?php
2. function sayHello($name="Sonoo"){
3. echo "Hello $name<br/>";
4. }
5. sayHello("Rajesh");
6. sayHello();//passing no value
7. sayHello("John");
8. ?>
```

Output:

```
Hello Rajesh
Hello Sonoo
Hello John
```

PHP Function: Returning Value

Let's see an example of PHP function that returns value.

File: functiondefaultarg.php

```
1. <?php
2. function cube($n){
3. return $n*$n*$n;
4. }
5. echo "Cube of 3 is: ".cube(3);
6. ?>
```

Output:

```
Cube of 3 is: 27
```

PHP header()

The header() is a **pre-defined network function** of PHP, which sends a **raw HTTP header** to a client. One important point to be noted about the header() function is that it must be called before sending any actual output.

The header() function sends an HTTP header in raw form to the client or browser. Before sending any other output, the HTTP functions manipulate the information sent by the web-server to the client or browser.

Syntax

```
1. void header (string $header, boolean $replace = TRUE, int $http_response_code)
```

Parameters

The header() function accepts three parameters, which are discussed below in detail:

\$header (mandatory)

The header parameter contains the header string to send. This function has two types of special header calls.

The first header starts with an "**HTTP/**" string, which locates the HTTP status code for sending.

The second special case of header starts with "**Location:**" header. It not only sends back the header to the browser, but it also provides a **REDIRECT (302) status code** to the browser, until the 201 or 3xx status code has already been set.

\$replace (optional)

This parameter is used for specifying whether a previous same header should be replaced by the header or add another header of same type. The \$replace is a **boolean** type **optional** parameter.

The default value is **TRUE**, which means that it replaces the previous same header. But you can bind several headers of same type if **FALSE** is passed as second argument.

http_response_code (optional)

The \$http_response_code is an **optional** parameter, which forces the HTTP response code to a specified value.

Note: The http_response_code parameter has an effect if the header is not blank.

Return Value

The PHP header() function does not return any value.

Changes

After **PHP version 5.1.2**, this function stops sending more than one header to prevent the header injection attacks. It allows only one header at a time.

Uses

- It changes the page location.
- It sets the time zone.
- It sends the STOP status.
- This function sets the caching control.
- It initiates the force download.

Examples

With the help of below examples, you can understand the actual working of header() function in runtime environment-

Example 1: Redirecting browser

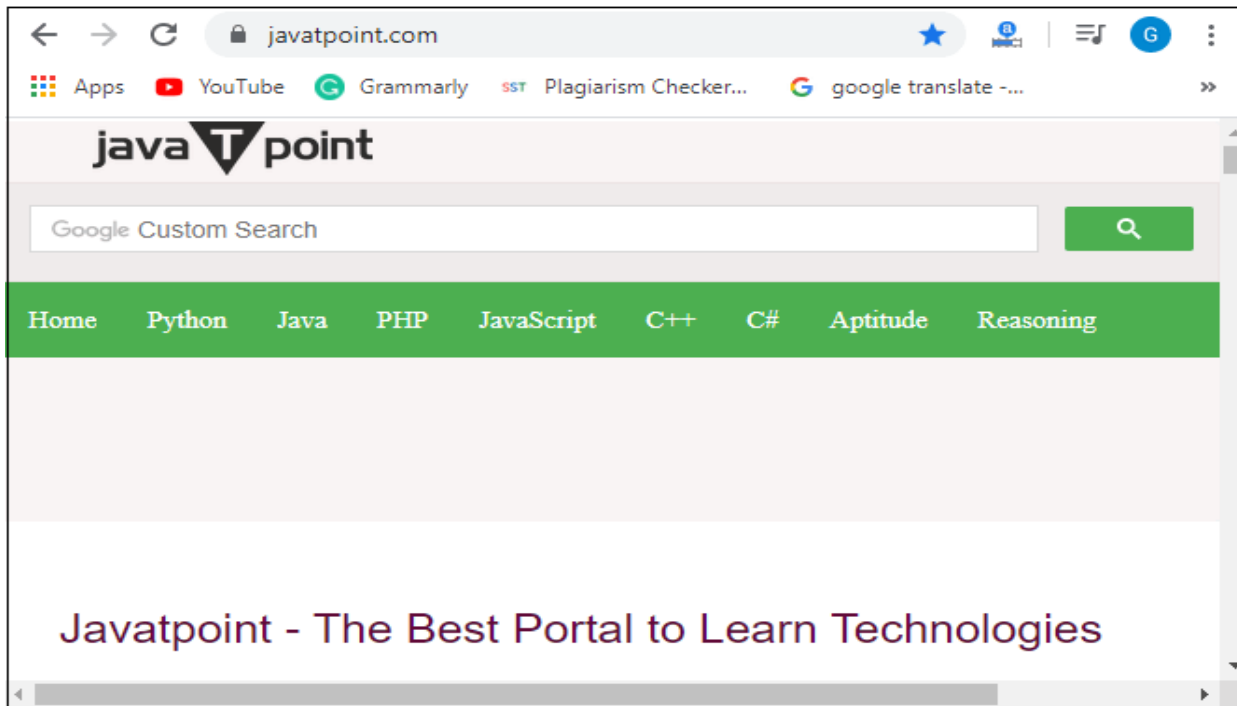
The following code will redirect your user to some another page.

```
1. <?php
2.     // This will redirect the user to the new location
3.     header('Location: http://www.javatpoint.com/');
4.
5.     //The below code will not execute after header
```

6. `exit;`
7. `?>`

Output

It will redirect to the new URL location, which is given in `header()` function of the above program, i.e., www.javatpoint.com. If any line of code is written after the `header()`, it will not execute.



Example 2: Redirection interval

The following code will **redirect** your user to another page **after 10 seconds**.

1. `<?php`
2. `// This will redirect after 10 seconds`
3. `header('Refresh: 10; url = http://www.javatpoint.com/');`
4. `exit;`
5. `?>`

Output

The output will be same as the example 1, but it will take 10 seconds to load.

Note: If any line of code is written after the `header()` function, it will not execute.

Example 3: Don't cache pages

By using the following code, you can prevent the browser to cache pages.

1. `<?php`
2. `// PHP program to describes header function`
3.
4. `// Set a past date`
5. `header("Expires: Tue, 03 March 2001 04:50:34 GMT");`
6. `header("Cache-Control: no-cache");`
7. `header("Pragma: no-cache");`
8. `?>`
9. `<html>`
10. `<body>`

```

11. <p>Hello Javatpoint!</p>
12.
13. <!-- PHP program to display
14. header list -->
15. <?php
16.     print_r(headers_list());
17. ?>
18. </body>
19. </html>

```

Output

```

Hello Javatpoint!
Array (
    [0] => X-Powered-By: PHP/7.3.13
    [1] => Expires: Tue, 03 March 2001 04:50:34 GMT
    [2] => Cache-Control: no-cache
    [3] => Pragma: no-cache
)

```

Absolute URI

Absolute URI is required by some older clients, which includes the hostname, scheme, and absolute path, whereas most contemporary clients accept relative URI as argument to **Location**. To make absolute URI, you can use `$SERVER['PHP_SELF']`, `$SERVER['HTTP_HOST']`, and `dirname()`.

Example 4

Create two php files, one of which for containing header file code and another for redirecting to a new page on the browser.

headercheck.php

```

1. <?php
2.     $host = $_SERVER['HTTP_HOST'];
3.     $uri = rtrim(dirname($_SERVER['PHP_SELF']), '/\');
4.     $newpage = 'welcome.php';
5.
6.     /* Redirect to a different page requested in the current directory*/
7.     header("Location: http://$host$uri/$newpage");
8.     exit;
9. ?>

```

welcome.php

```

1. <!-- welcome.php file redirect to new page -->
2. <html>
3.     <head>
4.         <title> Welcome page </title>
5.     </head>
6.     <body>
7.         <h1> <center> Welcome to javaTpoint </center> </h1>
8.     </body>

```

9. `</html>`
Output



Unit-4.0: MySQL Database

4.1 Introduction phpMyAdmin

4.1.1 Brief Introduction to
phpMyAdmin GUI tools

4.1.2 Create Database, table using
phpMyAdmin

4.1.3 SQL Operation on tables i.e.
insert, delete, update, drop

4.1.4 Import /Export table or
database.

4.2 MySQL Administration

4.2.1 Create MySQL User

4.2.2 Grant and revoke permission

4.3 MySQL Routine

4.3.1 Create and invoke a stored
Routine

4.3.2 Alter or drop stored routine

phpMyAdmin

phpMyAdmin

phpMyAdmin is an open-source software tool introduced on September 9, 1998, which is written in PHP. Basically, it is a third-party tool to manage the tables and data inside the database. phpMyAdmin supports various type of operations on MariaDB and MySQL. The main purpose of phpMyAdmin is to handle the administration of MySQL over the web.

This GUI tool manages a lot of data related operations such as creation, modification, deletion of data, change structure of table which are in the Databases. It can manage multiple databases at a time. It also performs other actions like browsing data, executing SQL queries along with managing indexes, users, and permissions. This also provides functionality like taking backups, doing search operations, import and export of data in different formats like SQL, CSV, XML, Excel sheets.

phpMyAdmin is the most trusted and user-friendly database managers and mostly used for web-based applications or programs. In the following article, we will be learning about the importance of the phpMyAdmin tool in the web world.

phpMyAdmin | Pre-requisites: To install phpMyAdmin software, you need a server running platform like Windows or Linux supports operating systems.

- Web Browser: You need a web browser interface to run the tool.
- PHP scripting language: You need a server side language.
- Apache Web server: You need a web server to store phpMyAdmin files.
- MySQL or MariaDB Database: You need a database to manage application data.

Note: XAMPP package installation is the easiest way to get phpMyAdmin tool.

phpMyAdmin | Features: Let us now look into some of the features of phpMyAdmin tool.

- Manages all user level permissions.
- Executes SQL queries, batch queries, triggers, events, stored procedures, functions to display relevant data results.
- It performs all data related operations like create, read, edit, delete, browse, change structure, schema and search.
- It also provide ways to import, export and load text file data.
- It export data in various formats like CSV, XML, PDF, Word document, Latex, Spreadsheet, Excel file and many more.
- It supports complex queries and bookmarking of SQL queries.
- It can also optimize, repair, change tables and views.
- It also supports InnoDB, foreign keys and MySQLi.
- It provides user friendly graphical interface to access website related data.
- It can provide PDF graphics layout of database.
- It is very flexible with different operating system.
- It can handle complex queries using Query by example.
- It provides multiple-server administration.

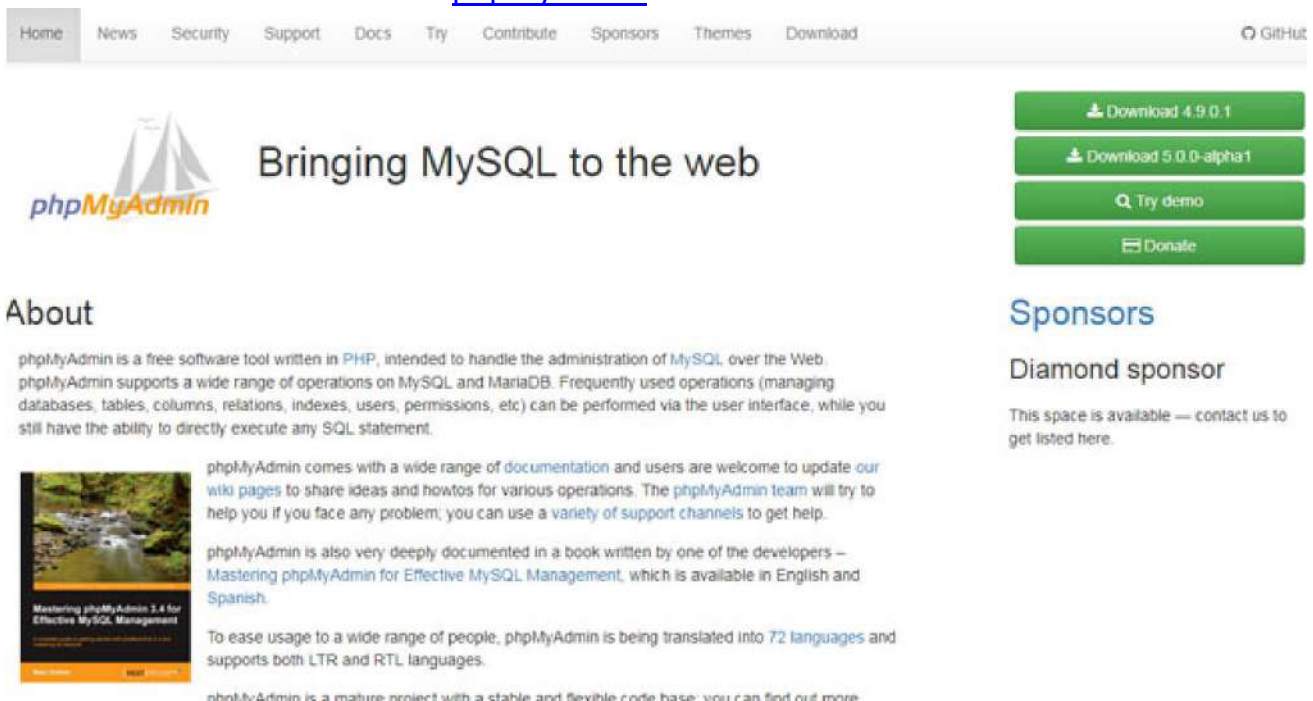
- It can control several servers at the same time.

phpMyAdmin | Advantages: Let us now look into some of the advantages of phpMyAdmin tool in brief.

- Its very simple to set up the tool.
- The GUI is very convenient and easy to understand for developers and users.
- As its a web-based tool, it can be accessed it from any computer system.
- Its a intuitive web based interface which runs on any server.
- It is advantageous over console as many tasks like cut, copy and run queries becomes very easy due to its GUI.
- It provides automatic MySQL backup facility.
- It don't require a network connectivity as its installed on the computer already having MySQL server.
- It provides maximum level of data security and efficiency for its client.
- It is supported by multi-language community.

phpMyAdmin | Configuration:

- Download the latest version of [phpMyAdmin](https://www.phpmyadmin.net/).



Home News Security Support Docs Try Contribute Sponsors Themes Download [GitHub](#)

phpMyAdmin Bringing MySQL to the web

About

phpMyAdmin is a free software tool written in [PHP](#), intended to handle the administration of [MySQL](#) over the Web. phpMyAdmin supports a wide range of operations on MySQL and MariaDB. Frequently used operations (managing databases, tables, columns, relations, indexes, users, permissions, etc) can be performed via the user interface, while you still have the ability to directly execute any SQL statement.

phpMyAdmin comes with a wide range of [documentation](#) and users are welcome to update [our wiki pages](#) to share ideas and howtos for various operations. The [phpMyAdmin team](#) will try to help you if you face any problem; you can use a [variety of support channels](#) to get help.

phpMyAdmin is also very deeply documented in a book written by one of the developers – [Mastering phpMyAdmin for Effective MySQL Management](#), which is available in English and Spanish.

To ease usage to a wide range of people, phpMyAdmin is being translated into [72 languages](#) and supports both LTR and RTL languages.

phpMyAdmin is a mature project with a stable and flexible code base; you can find out more

Sponsors

Diamond sponsor

This space is available — contact us to get listed here.

- Once the files are extracted during the download process, they are stored in location C:\xampp\phpMyAdmin
- The configuration settings can be changed in config.inc.php file which is present in the above location. Other system settings can also be changed in the file.
- Administrators have full rights for creating new users, remove users and modify privileges for old users.
- For a specific database, you can define the permissions of any user and manage accounts of user groups.

Note: The configuration settings are used to customize the web interface.

phpMyAdmin | Logging in: Whenever a user logs in to phpMyAdmin server with the username and password, these values are passed directly to MySQL server. For logging in, you need the valid credentials of MySQL users.

- Database server
- Database username
- Database password



Welcome to phpMyAdmin

Language

English ▼

Log in 

Username:

Password:

Go

Once all the credentials are entered correctly, you can see the phpMyAdmin home page in your browser. When you are in phpMyAdmin page, the center section is the default General Settings and Appearance Settings and in the right side, you can see Database server and Web server information.

General Settings
Server connection collation  utf8_general_ci ▼

Appearance Settings
Language  English ▼
Theme: pmahomme ▼
Font size: 82% ▼
[More settings](#)

Database server

- Server: internal-db.s84318.gridserver.com via TCP/IP
- Software: Percona Server
- Software version: 5.6.39-83.1 - Percona Server (GPL), Release 83.1, Revision da5a1c2923f
- Protocol version: 10
- User: db84318_ppart@72.47.228.16
- Server charset: UTF-8 Unicode (utf8)

Web server

- Apache/2.2.34
- Database client version: libmysql - mysqlnd 5.0.11-dev - 20120503 - \$Id: 76b08b24596e1244553bd41fc93cccd5bac2fe7a \$
- PHP extension: mysql 

phpMyAdmin

- Version information: 3.5.2
- [Documentation](#)
- [Wiki](#)
- [Official Homepage](#)
- [Contribute](#)
- [Get support](#)
- [List of changes](#)

You can adjust language, themes, font size under the Appearance Settings section. You can also change Database server information as needed.

All the list of databases can be seen on the left-hand side of the screen. You can select the database, you want to work with out of the list. Once it is expanded, all the tables in the selected database can be viewed.



phpMyAdmin tool allows you to check the MySQL server status by accessing the Status tab. For checking status, remember NOT to select any specific database.

The image shows the 'Status' tab in phpMyAdmin. At the top, there's a navigation bar with tabs: 'Databases', 'SQL', 'Status' (selected), 'Users', 'Export', 'Import', 'Settings', 'Replication', and 'Variables'. Below the navigation bar, there are sub-tabs: 'Server', 'Query statistics', 'All status variables', 'Monitor', and 'Advisor'. The 'Server' sub-tab is selected. The main content area shows the following information:
Network traffic since startup: 947.2 KiB
This MySQL server has been running for 12 days, 3 hours, 25 minutes and 13 seconds. It started up on Jul 12, 2019 at 06:55 AM.

Traffic	per hour
Received	288.1 KiB
Sent	659.1 KiB
Total	947.2 KiB

Connections	per hour	%
max. concurrent connections	2	---
Failed attempts	0	0%
Aborted	0	0%
Total	308.83	100.00%

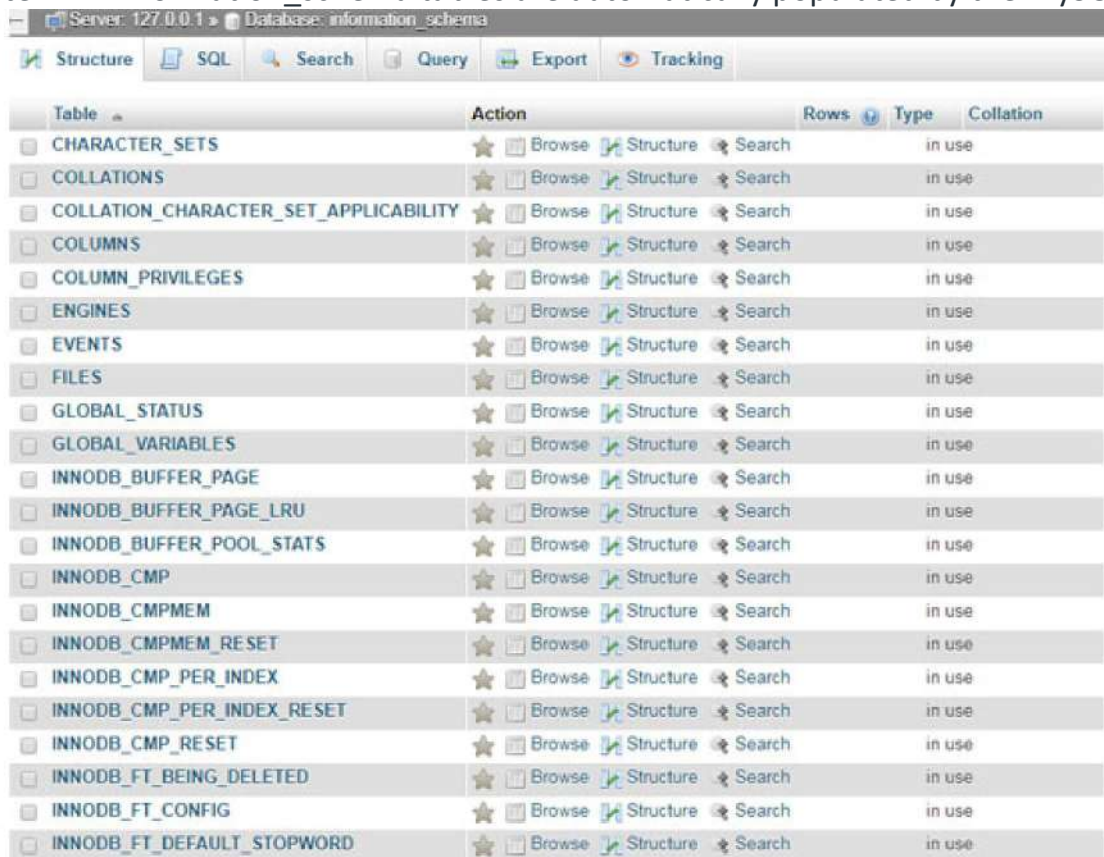
Processes	ID	User	Host	Database	Command	Time	Status	SQL query
Kill	88	pma	localhost:58398	None	Sleep	0	---	---
Kill	89	root	localhost:58399	mysql	Query	0	init	---

At the bottom right, there's a link that says 'SHOW PROCESSLIST'.

Here, you can find information regarding all the traffic your server has Received and Sent while the MySQL server been running. It also shows the maximum concurrent connections, and Failed attempts. If you are using a shared hosting plan, chances are your numbers won't make much sense. All these information helps in determining the amount of bandwidth consumed as you are not the only one to use the server.

phpMyAdmin | Access of Database information: You can see information_schema database in the left side of the screen. It is the database which stores all the information of all other databases like name of databaseS, name of tables, column data type, access privileges and so on. It is a built-in database created with the purpose of giving information about the whole database

system. In information_schema tables are automatically populated by the MySQL DB server.



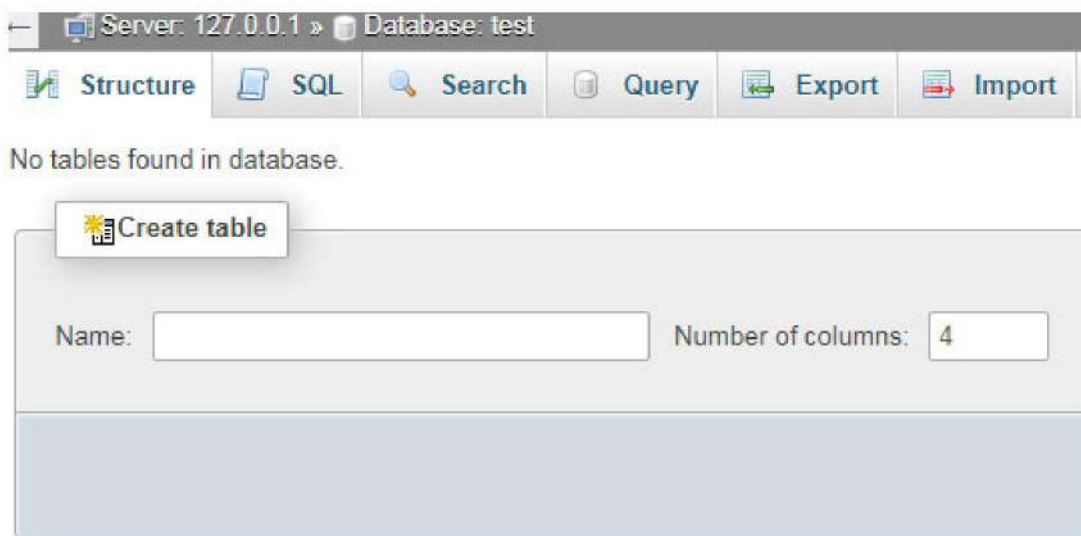
The screenshot shows the MySQL Server interface with the 'information_schema' database selected. The 'Table' tab is active, displaying a list of system tables. Each table has a checkbox, a star icon, and links for 'Browse', 'Structure', and 'Search'. The 'Rows', 'Type', and 'Collation' columns are also visible.

Table	Action	Rows	Type	Collation
☐ CHARACTER_SETS	★ Browse Structure Search			in use
☐ COLLATIONS	★ Browse Structure Search			in use
☐ COLLATION_CHARACTER_SET_APPLICABILITY	★ Browse Structure Search			in use
☐ COLUMNS	★ Browse Structure Search			in use
☐ COLUMN_PRIVILEGES	★ Browse Structure Search			in use
☐ ENGINES	★ Browse Structure Search			in use
☐ EVENTS	★ Browse Structure Search			in use
☐ FILES	★ Browse Structure Search			in use
☐ GLOBAL_STATUS	★ Browse Structure Search			in use
☐ GLOBAL_VARIABLES	★ Browse Structure Search			in use
☐ INNODB_BUFFER_PAGE	★ Browse Structure Search			in use
☐ INNODB_BUFFER_PAGE_LRU	★ Browse Structure Search			in use
☐ INNODB_BUFFER_POOL_STATS	★ Browse Structure Search			in use
☐ INNODB_CMP	★ Browse Structure Search			in use
☐ INNODB_CMPMEM	★ Browse Structure Search			in use
☐ INNODB_CMPMEM_RESET	★ Browse Structure Search			in use
☐ INNODB_CMP_PER_INDEX	★ Browse Structure Search			in use
☐ INNODB_CMP_PER_INDEX_RESET	★ Browse Structure Search			in use
☐ INNODB_CMP_RESET	★ Browse Structure Search			in use
☐ INNODB_FT_BEING_DELETED	★ Browse Structure Search			in use
☐ INNODB_FT_CONFIG	★ Browse Structure Search			in use
☐ INNODB_FT_DEFAULT_STOPWORD	★ Browse Structure Search			in use

phpMyAdmin | Create database: A user can create a database in phpMyAdmin server. Just click on the Databases tab, you have a Create database input option. Give a relevant name for the database in the entry box and click the Create button. The phpMyAdmin will create tables which will be added under the newly created database.

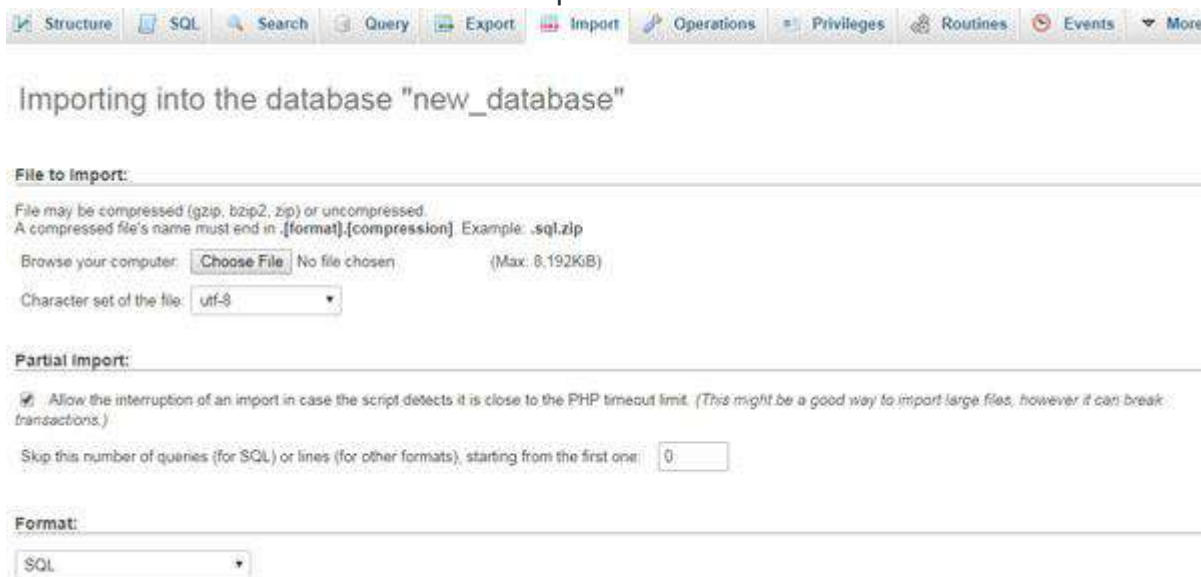


You can add as many numbers of tables as required in one particular selected database by using the section Create table and then the select number of columns needed for any table.



The screenshot shows the phpMyAdmin interface with the 'Database: test' selected. The top navigation bar includes tabs for Structure, SQL, Search, Query, Export, and Import. Below the navigation bar, a message states 'No tables found in database.' A 'Create table' button is visible. Below this button, there is a form with two input fields: 'Name:' and 'Number of columns:'. The 'Number of columns:' field is set to '4'.

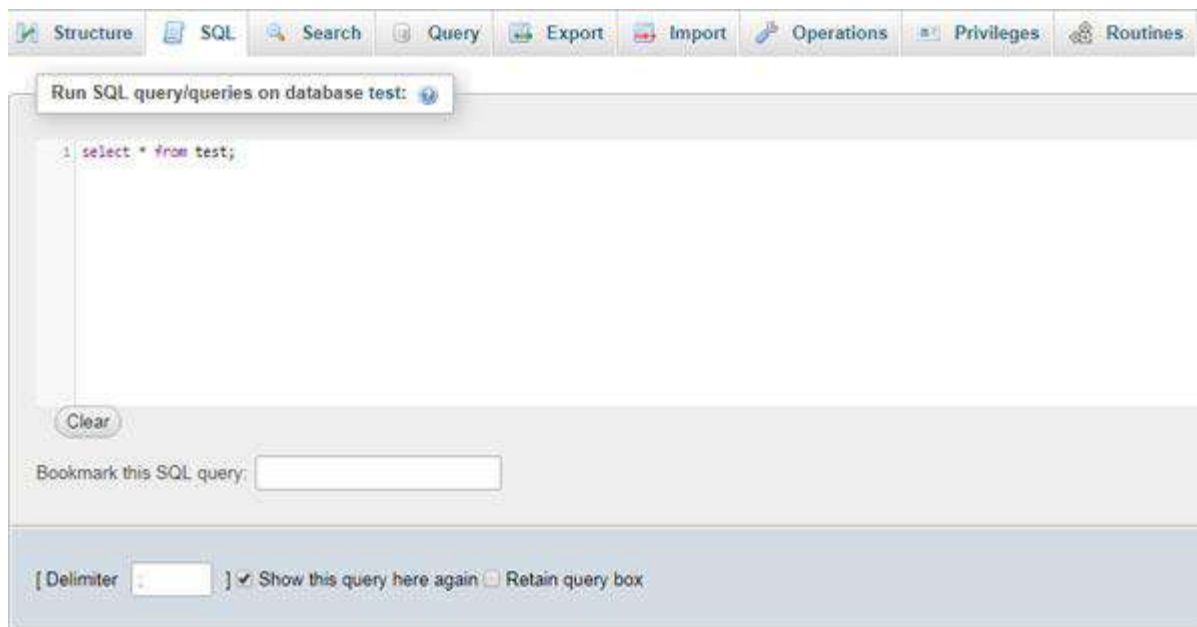
The phpMyAdmin will give an option to upload an SQL file to your server. Click on the Import tab of the top screen and it will allow you to choose a file from any location. This helps in creating database or tables as written in the imported file.



The screenshot shows the 'Import' tab selected in the phpMyAdmin interface. The title is 'Importing into the database "new_database"'. Under the 'File to Import:' section, there is a text area for file information and a 'Choose File' button. Below this, there is a 'Character set of the file:' dropdown menu set to 'utf-8'. Under the 'Partial Import:' section, there is a checkbox for 'Allow the interruption of an import in case the script detects it is close to the PHP timeout limit.' and a text input field for 'Skip this number of queries (for SQL) or lines (for other formats), starting from the first one:' set to '0'. Under the 'Format:' section, there is a dropdown menu set to 'SQL'.

Note: In the same way, you can also export any table or database as a file by using Export tab. phpMyAdmin | Execute a SQL query: Let us look into the steps to execute a SQL query on a selected database.

- In the left side of the screen, select the database to work with. The Structure of the database can be seen after the selection.
- Click SQL tab from top bar to open editor in which you can write the queries.
- Write the query into the SQL editor.
- You can also write multiple queries, with semicolon (;) separating different queries.
- Click Go button to execute the query. If the query is executed successfully, you will get a success message.



Another way to run a SQL query in the database is by using Query tab. Using this you can give many search conditions, query more than one table and also sort the results. You can even easily pick all the columns needed for MySQL query by using Column drop down menus and Show checkbox.

For sorting the result set in a query in an ascending or descending order, you can use the Sort dropdown. You can also enter the criteria for the search query. The Ins and Del checkboxes or Add/Delete criteria rows options are provided to add or delete text rows for any search conditions. The logical operators And or the Or can also be used to define relations between the fields in the Modify section. After entering all the relevant conditions, you need to click on Update Query button for doing the SQL operation. Next step is to select all the tables from Use Tables section used in the query and then click on Submit Query.

Switch to [visual builder](#)

Saved bookmarked search: New bookmark ▼ Create bookmark

Column: ▼ ▼ ▼

Sort: ▼ ▼ ▼

Show: ☐ ☐ ☐

Criteria: ☐ ☐ ☐

Ins ☐ And: ☐ Del ☐ Or: ☐ And: ☐ Ins ☐ Del ☐

Modify: Or: ☐ And: ☐ Ins ☐ Del ☐ Or: ☐ And: ☐ Ins ☐ Del ☐ Or: ☐ And: ☐ Ins ☐ Del ☐

Add/Delete criteria rows: 0 ▼ Add/Delete columns: 0 ▼ Update Query

Use Tables

category
igk_commentmeta
igk_comments
igk_links
igk_options
igk_postmeta
igk_posts

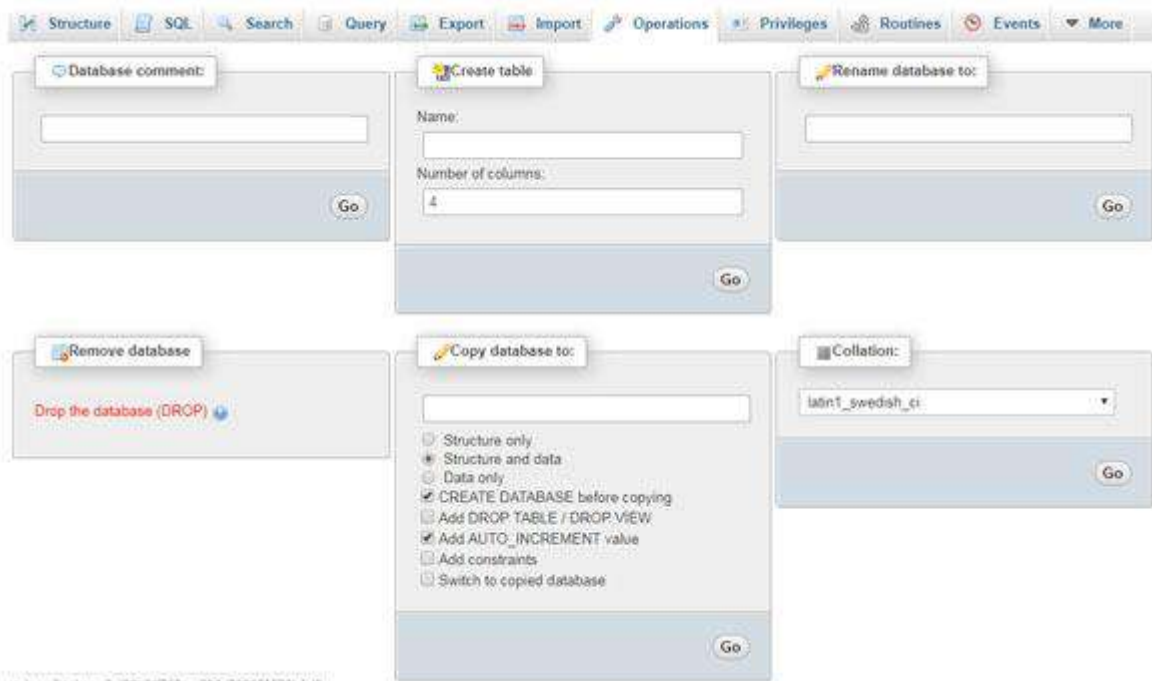
Update Query

SQL query on database recipedb:

Submit Query

phpMyAdmin | Operations: In the phpMyAdmin tool, different operations can be performed on the database or on a separate table. Select the database for which you want to perform any operation and click the Operations tab. It is allowed to create a new table under the selected database by using Create table section. You can also rename the selected database by using Rename database to section. Similarly you also have Remove database to delete database

and Copy database to sections which can be used as required for any SQL operation.



phpMyAdmin | Generate a search query: We can also generate a search query using phpMyAdmin GUI, rather than writing the search query manually for a selected table. This can be done by clicking the Search tab in the top menu of the screen.

phpMyAdmin | Insert query: We can also insert query using phpMyAdmin GUI, rather than manually writing it. Select the table for which insert query will be executed. Then click the Insert tab in the top menu of the screen. Once the relevant values are entered, the new record will be inserted in the table. The newly inserted record can be viewed using the Browse tab under the selected table name.

MySQL - Administration

MySQL Server is the program that mainly executes the SQL queries in the database system. Hence it becomes necessary to optimize the working of this server. The general MySQL administration usually includes concepts like:

- Starting and Stopping the Server
- User Security
- Database Maintenance
- Backup & Restore

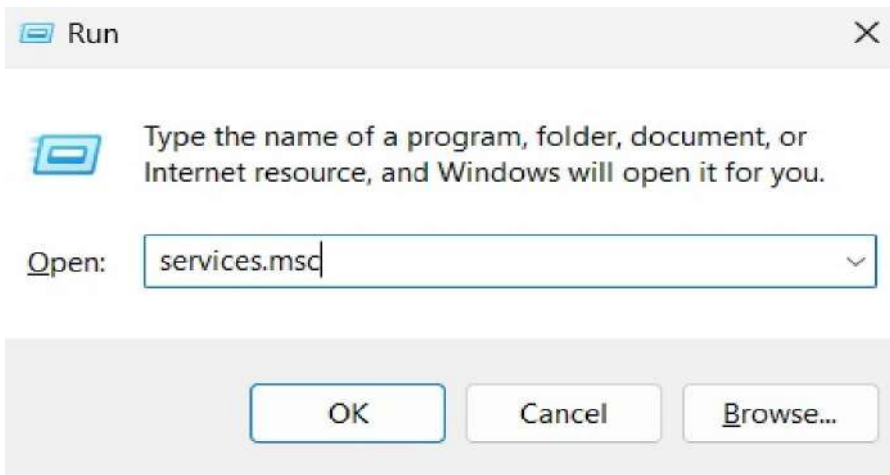
Start MySQL Server

We need to first start the MySQL server on the device in order to use it. One way to do so, is by executing the following command on the command prompt (run as an administrator) –

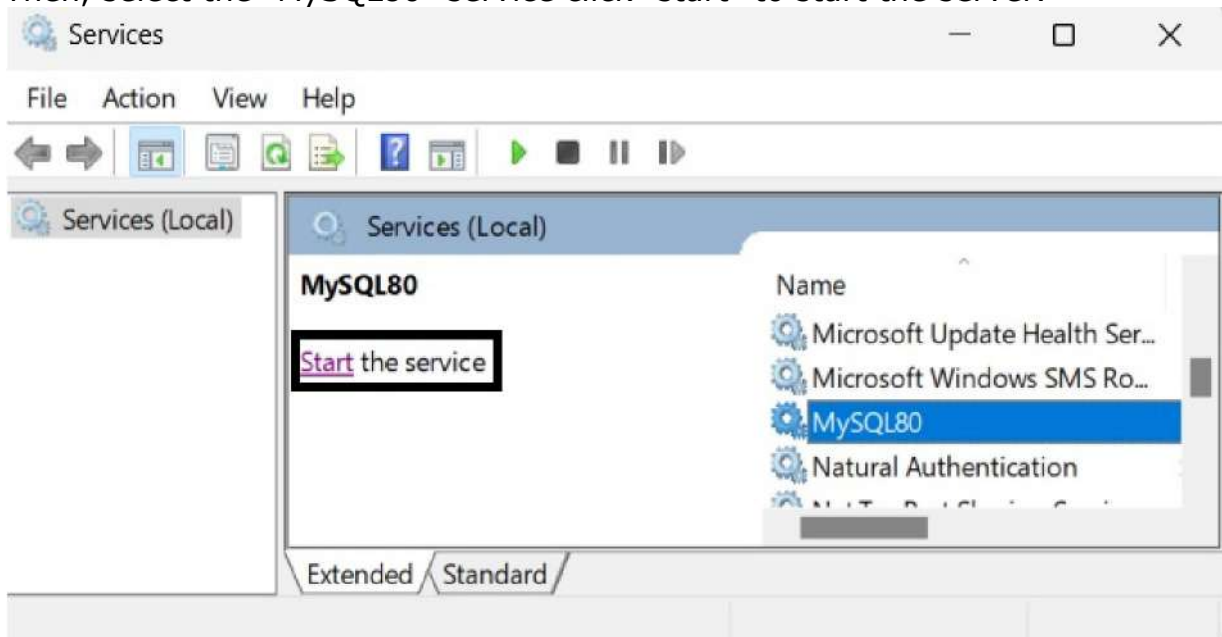
```
mysqld
```

We can also start the server by going through the services provided by the Windows and follow the steps below –

- Open the 'Run' Window using the 'Windows+R' shortcut and run 'services.msc' through it.



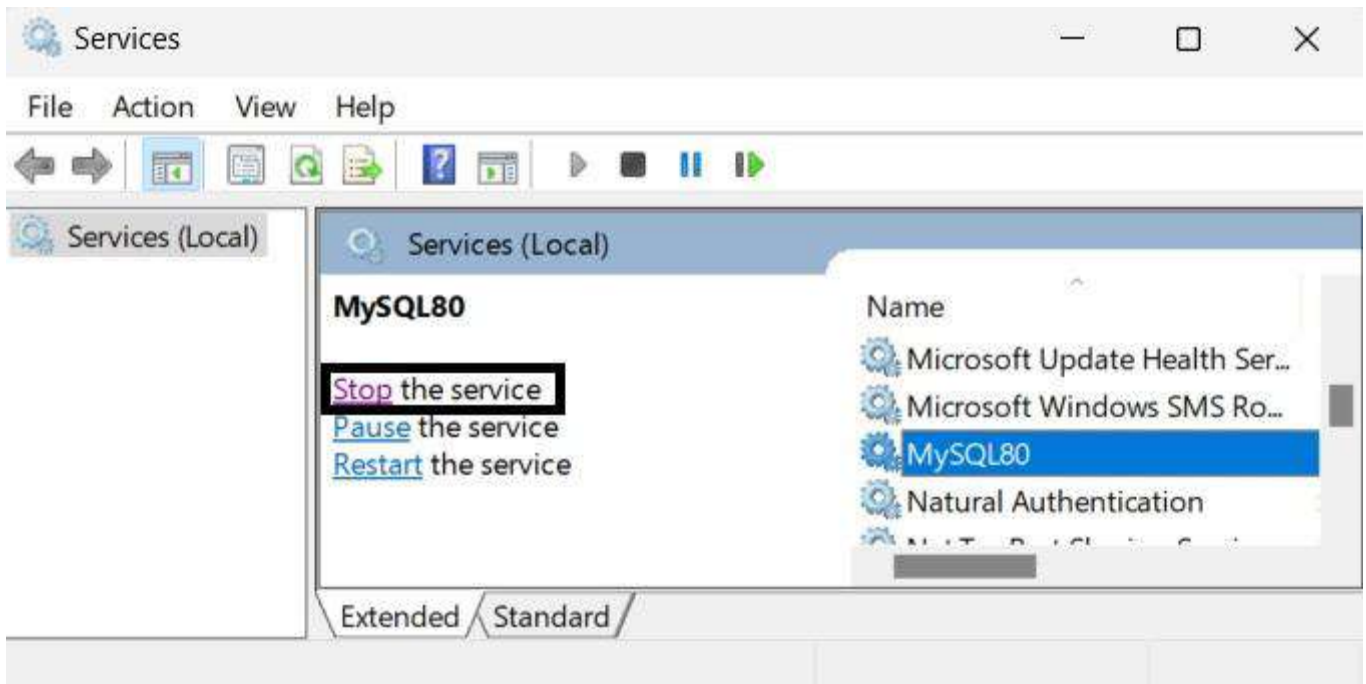
- Then, select the "MySQL80" service click "start" to start the server.



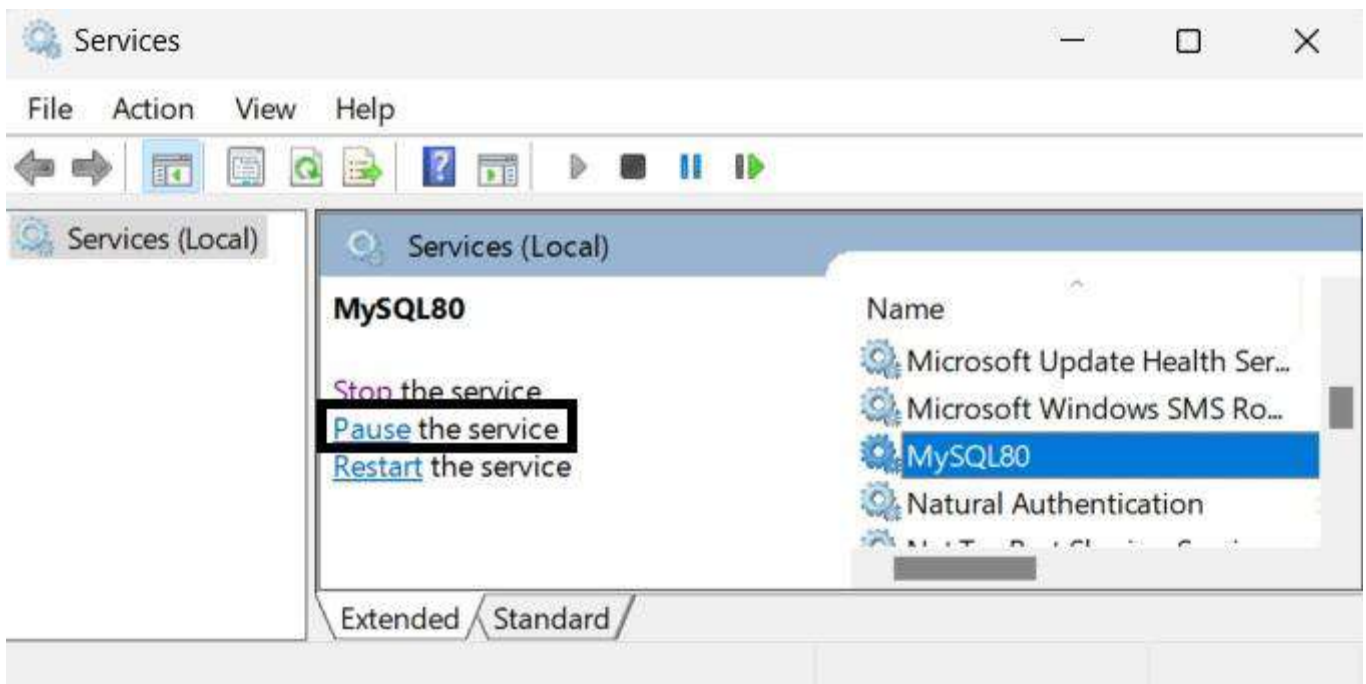
Stop, Pause, Restart MySQL Server

Now, if you want to pause, stop or restart an already running MySQL server, then you can do it by opening the Windows Services and selecting the desired action –

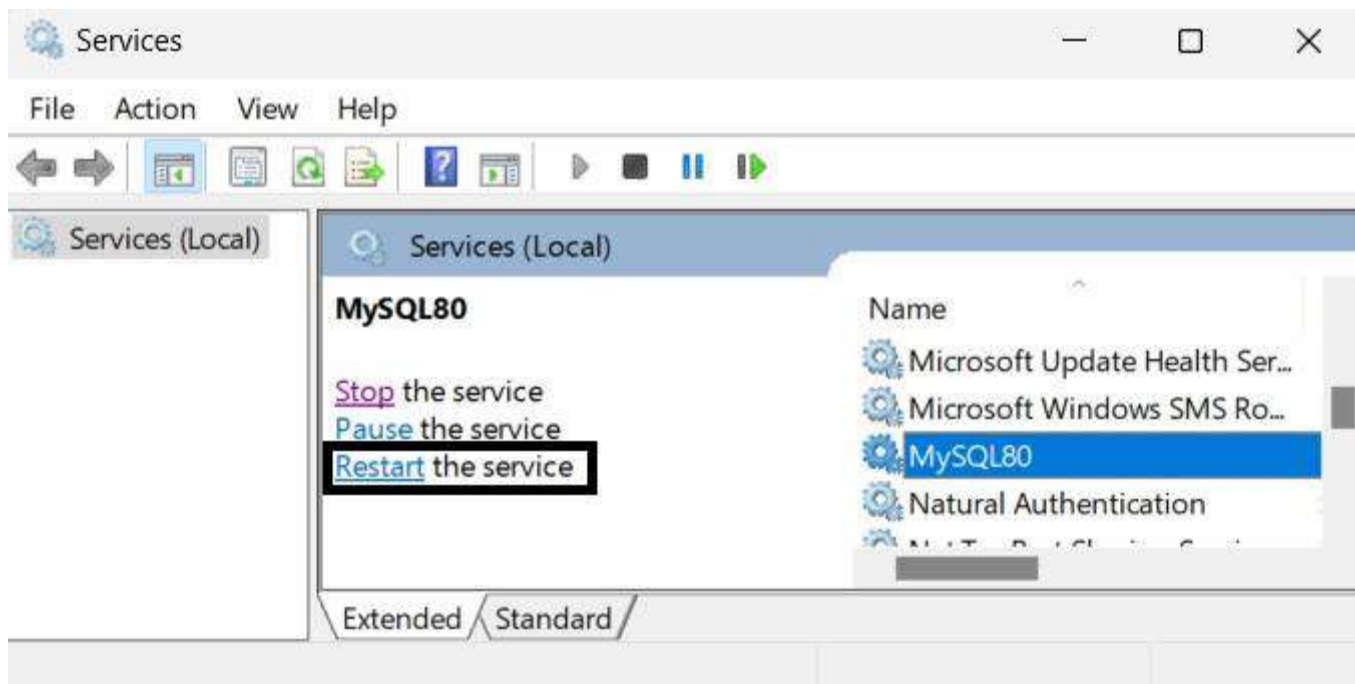
To stop the MySQL Server, select the 'stop' option as shown in the image below –



To pause the MySQL Server, select the 'pause' option as shown in the image below –



We can also restart the MySQL server as needed, by selecting the 'restart' option as shown below –



Learn MySQL in-depth with real-world projects through our MySQL certification course. Enroll and become a certified expert to boost your career.

Setting Up a MySQL User Account

For adding a new user to MySQL, you just need to add a new entry to the **user** table in the database **mysql**.

In the following example, we are creating a new user **guest** with the password **guest123** on the 'localhost'. We are also granting all privileges required to executing SQL queries –

```
CREATE USER 'guest'@'localhost' IDENTIFIED BY 'guest123';
```

Now, execute the FLUSH PRIVILEGES statement. This tells the server to reload the grant tables. If you don't use it, then you won't be able to connect to MySQL using the new user account at least until the server is rebooted.

```
FLUSH PRIVILEGES;
```

Finally, you need to grant all privileges to this new user to execute SQL queries.

```
GRANT ALL PRIVILEGES ON * . * TO 'sample'@'localhost';
```

You can also specify other privileges to a new user by setting the values of following columns in 'user' table to 'Y' using the UPDATE query.

- Select_priv
- Insert_priv
- Update_priv
- Delete_priv
- Create_priv
- Drop_priv

- Reload_priv
- Shutdown_priv
- Process_priv
- File_priv
- Grant_priv
- References_priv
- Index_priv
- Alter_priv

NOTE – MySQL does not terminate a command until you give a semi colon (;) at the end of the SQL command.

The /etc/my.cnf File Configuration

In most of the cases, you should not touch this file. By default, it will have the following entries –

[mysqld]

datadir = /var/lib/mysql

socket = /var/lib/mysql/mysql.sock

[mysql.server]

user = mysql

basedir = /var/lib

[safe_mysqld]

err-log = /var/log/mysqld.log

pid-file = /var/run/mysqld/mysqld.pid

Here, you can specify a different directory for the error log, otherwise you should not change any entry in this table.



Administrative MySQL Commands

Here is the list of the important MySQL commands, which you will use time to time to work with MySQL database –

- **USE database_name** – This will be used to select a database in the MySQL.
- **SHOW DATABASES** – Lists out the databases that are accessible by the MySQL DBMS.
- **SHOW TABLES** – Displays the list of the tables in the current database.
- **SHOW COLUMNS FROM table_name**: Shows the attributes, types of attributes, key information, whether NULL is permitted, defaults, and other information for a table.
- **SHOW INDEX FROM table_name** – Presents the details of all indexes on the table, including the PRIMARY KEY.
- **SHOW TABLE STATUS LIKE table_name\G** – Reports details of the MySQL DBMS performance and statistics.

MySQL Routines

MySQL supports stored routines (procedures and functions). A stored routine is a set of SQL statements that can be stored in the server. Once this has been done, clients don't need to keep reissuing the individual statements but can refer to the stored routine instead.

MySQL Stored Procedure

A procedure (often called a stored procedure) is a collection of pre-compiled SQL statements stored inside the database. It is a subroutine or a subprogram in the regular computing language. A procedure always contains a name, parameter lists, and SQL statements.

Stored Procedure Features

- Stored Procedure increases the performance of the applications. Once stored procedures are created, they are compiled and stored in the database.
- Stored procedure reduces the traffic between application and database server. Because the application has to send only the stored procedure's name and parameters instead of sending multiple SQL statements.
- Stored procedures are reusable and transparent to any applications.
- A procedure is always secure. The database administrator can grant permissions to applications that access stored procedures in the database without giving any permissions on the database tables.

How to create a procedure?

The following syntax is used for creating a stored procedure in MySQL. It can return one or more value through parameters or sometimes may not return at all. By default, a procedure is associated with our current database. But we can also create it into another database from the current database by specifying the name as **database_name.procedure_name**. See the complete syntax:

1. DELIMITER &&
2. **CREATE PROCEDURE** procedure_name [[IN | **OUT** | INOUT] parameter_name datatype [, parameter data type]]]
3. **BEGIN**
4. Declaration_section
5. Executable_section
6. **END** &&
7. DELIMITER ;

Parameter Explanations

The procedure syntax has the following parameters:

Parameter Name	Descriptions
procedure_name	It represents the name of the stored procedure.
parameter	It represents the number of parameters. It can be one or more than one.
Declaration_section	It represents the declarations of all variables.
Executable_section	It represents the code for the function execution.

MySQL procedure parameter has one of three modes:

IN parameter

It is the default mode. It takes a parameter as input, such as an attribute. When we define it, the calling program has to pass an argument to the stored procedure. This parameter's value is always protected.

OUT parameters

It is used to pass a parameter as output. Its value can be changed inside the stored procedure, and the changed (new) value is passed back to the calling program. It is noted that a procedure cannot access the OUT parameter's initial value when it starts.

INOUT parameters

It is a combination of IN and OUT parameters. It means the calling program can pass the argument, and the procedure can modify the INOUT parameter, and then passes the new value back to the calling program.

How to call a stored procedure?

We can use the **CALL statement** to call a stored procedure. This statement returns the values to its caller through its parameters (IN, OUT, or INOUT). The following syntax is used to call the stored procedure in MySQL:

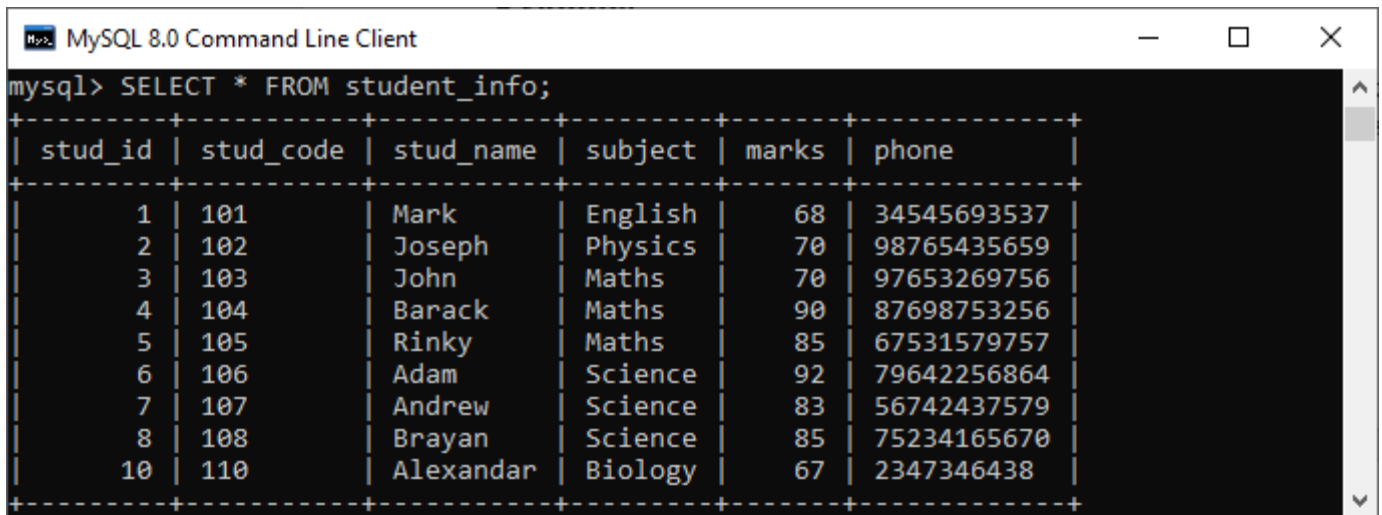
1. `CALL procedure_name ( parameter(s))`

Example

Let us understand how to create a procedure in MySQL through example. First, we need to select a database that will store the newly created procedure. We can select the database using the below statement:

1. `mysql> USE database_name;`

Suppose this database has a table named **student_info** that contains the following data:



```
mysql> SELECT * FROM student_info;
```

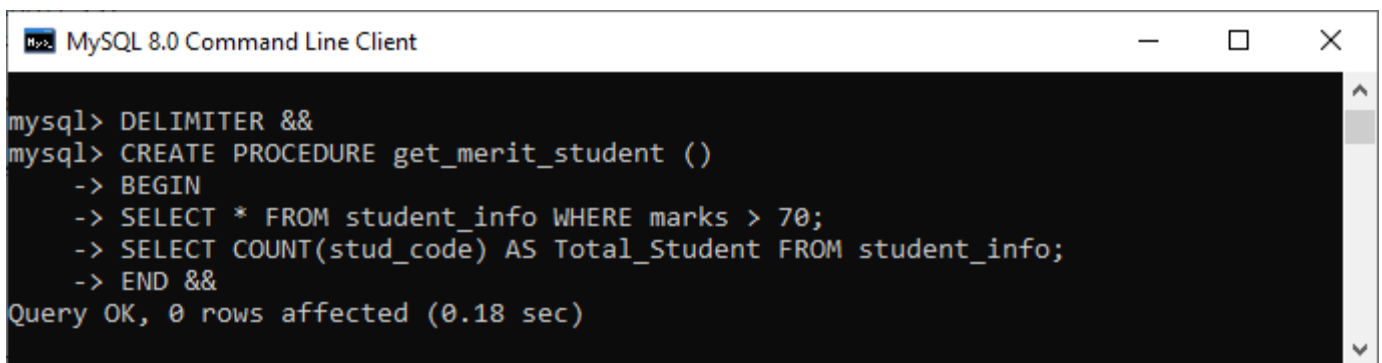
stud_id	stud_code	stud_name	subject	marks	phone
1	101	Mark	English	68	34545693537
2	102	Joseph	Physics	70	98765435659
3	103	John	Maths	70	97653269756
4	104	Barack	Maths	90	87698753256
5	105	Rinky	Maths	85	67531579757
6	106	Adam	Science	92	79642256864
7	107	Andrew	Science	83	56742437579
8	108	Brayan	Science	85	75234165670
10	110	Alexandar	Biology	67	2347346438

Procedure without Parameter

Suppose we want to **display all records of this table whose marks are greater than 70** and count all the table rows. The following code creates a procedure named **get_merit_students**:

1. DELIMITER &&
2. **CREATE PROCEDURE** get_merit_student ()
3. **BEGIN**
4. **SELECT * FROM** student_info **WHERE** marks > 70;
5. **SELECT COUNT**(stud_code) **AS** Total_Student **FROM** student_info;
6. **END** &&
7. DELIMITER ;

If this code executed successfully, we would get the below output:



```
mysql> DELIMITER &&
mysql> CREATE PROCEDURE get_merit_student ()
-> BEGIN
-> SELECT * FROM student_info WHERE marks > 70;
-> SELECT COUNT(stud_code) AS Total_Student FROM student_info;
-> END &&
Query OK, 0 rows affected (0.18 sec)
```

Let us call the procedure to verify the output:

1. mysql> CALL get_merit_student();
It will give the output as follows:


```

MySQL 8.0 Command Line Client
mysql> CALL get_merit_student();
+-----+-----+-----+-----+-----+-----+
| stud_id | stud_code | stud_name | subject | marks | phone |
+-----+-----+-----+-----+-----+-----+
| 4 | 104 | Barack | Maths | 90 | 87698753256 |
| 5 | 105 | Rinky | Maths | 85 | 67531579757 |
| 6 | 106 | Adam | Science | 92 | 79642256864 |
| 7 | 107 | Andrew | Science | 83 | 56742437579 |
| 8 | 108 | Brayan | Science | 85 | 75234165670 |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

+-----+
| Total_Student |
+-----+
| 9 |
+-----+
1 row in set (0.03 sec)

```

Procedures with IN Parameter

In this procedure, we have used the IN parameter as 'var1' of integer type to accept a number from users. Its body part fetches the records from the table using a **SELECT statement** and returns only those rows that will be supplied by the user. It also returns the total number of rows of the specified table. See the procedure code:

1. DELIMITER &&
2. **CREATE PROCEDURE** get_student (IN var1 INT)
3. **BEGIN**
4. **SELECT * FROM** student_info LIMIT var1;
5. **SELECT COUNT**(stud_code) **AS** Total_Student **FROM** student_info;
6. **END &&**
7. DELIMITER ;

After successful execution, we can call the procedure as follows:

1. mysql> CALL get_student(4);
We will get the below output:

```

MySQL 8.0 Command Line Client
mysql> CALL get_student(4);
+-----+-----+-----+-----+-----+-----+
| stud_id | stud_code | stud_name | subject | marks | phone |
+-----+-----+-----+-----+-----+-----+
| 1 | 101 | Mark | English | 68 | 34545691537 |
| 2 | 102 | Joseph | Physics | 78 | 98765435659 |
| 3 | 103 | John | Maths | 70 | 97651269756 |
| 4 | 104 | Barack | Maths | 90 | 87698753256 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

+-----+
| Total_Student |
+-----+
| 9 |
+-----+

```

Procedures with OUT Parameter

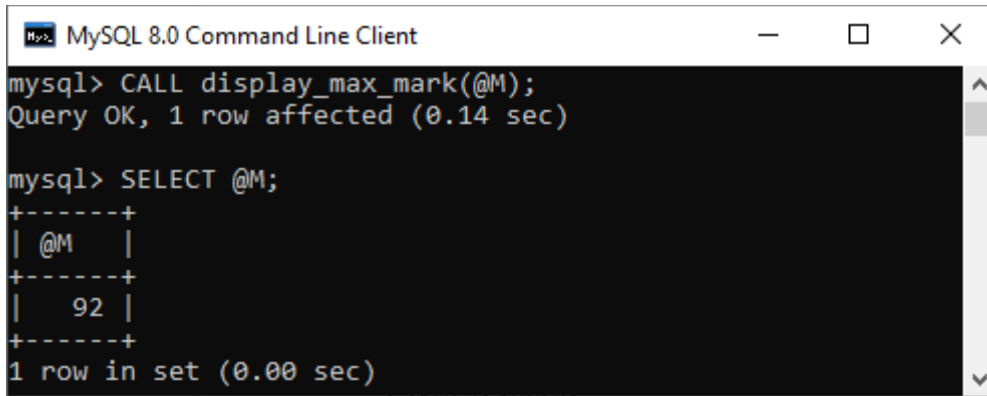
In this procedure, we have used the OUT parameter as the 'highestmark' of integer type. Its body part fetches the maximum marks from the table using a **MAX() function**. See the procedure code:

1. DELIMITER &&
2. **CREATE PROCEDURE** display_max_mark (**OUT** highestmark INT)
3. **BEGIN**

4. **SELECT MAX**(marks) **INTO** highestmark **FROM** student_info;
5. **END** &&
6. **DELIMITER** ;

This procedure's parameter will get the highest marks from the **student_info** table. When we call the procedure, the OUT parameter tells the database systems that its value goes out from the procedures. Now, we will pass its value to a session variable **@M** in the CALL statement as follows:

1. mysql> **CALL** display_max_mark(@M);
 2. mysql> **SELECT** @M;
- Here is the output:



```
mysql> CALL display_max_mark(@M);
Query OK, 1 row affected (0.14 sec)

mysql> SELECT @M;
+-----+
| @M    |
+-----+
| 92    |
+-----+
1 row in set (0.00 sec)
```

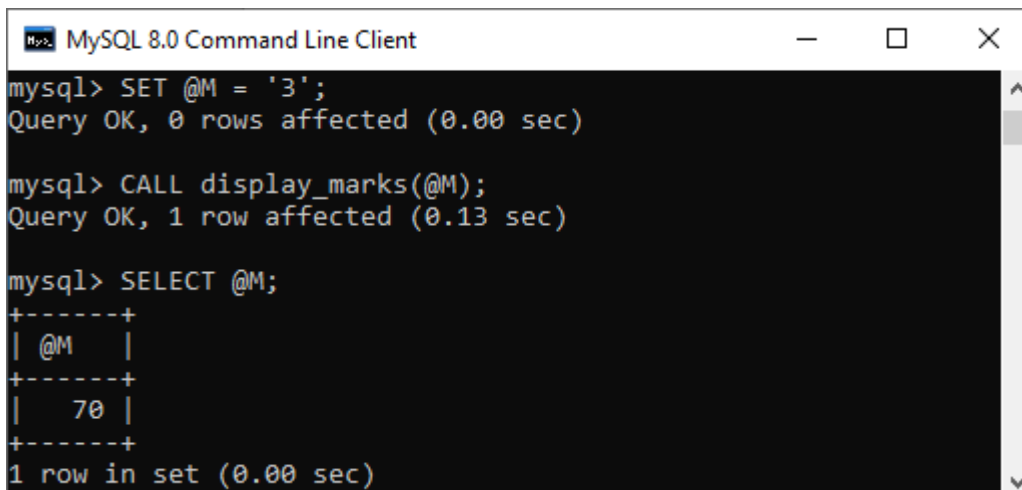
Procedures with INOUT Parameter

In this procedure, we have used the INOUT parameter as '**var1**' of integer type. Its body part first fetches the marks from the table with the specified **id** and then stores it into the same variable **var1**. The **var1** first acts as the IN parameter and then OUT parameter. Therefore, we can call it the INOUT parameter mode. See the procedure code:

1. **DELIMITER** &&
2. **CREATE PROCEDURE** display_marks (INOUT var1 **INT**)
3. **BEGIN**
4. **SELECT** marks **INTO** var1 **FROM** student_info **WHERE** stud_id = var1;
5. **END** &&
6. **DELIMITER** ;

After successful execution, we can call the procedure as follows:

1. mysql> **SET** @M = '3';
 2. mysql> **CALL** display_marks(@M);
 3. mysql> **SELECT** @M;
- We will get the below output:



```
mysql> SET @M = '3';
Query OK, 0 rows affected (0.00 sec)

mysql> CALL display_marks(@M);
Query OK, 1 row affected (0.13 sec)

mysql> SELECT @M;
+-----+
| @M    |
+-----+
| 70    |
+-----+
1 row in set (0.00 sec)
```

How to show or list stored procedures in MySQL?

When we have several procedures in the MySQL server, it is very important to list all procedures. It is because sometimes the procedure names are the same in many databases. In that case, this query is very useful. We can list all procedure stored on the current MySQL server as follows:

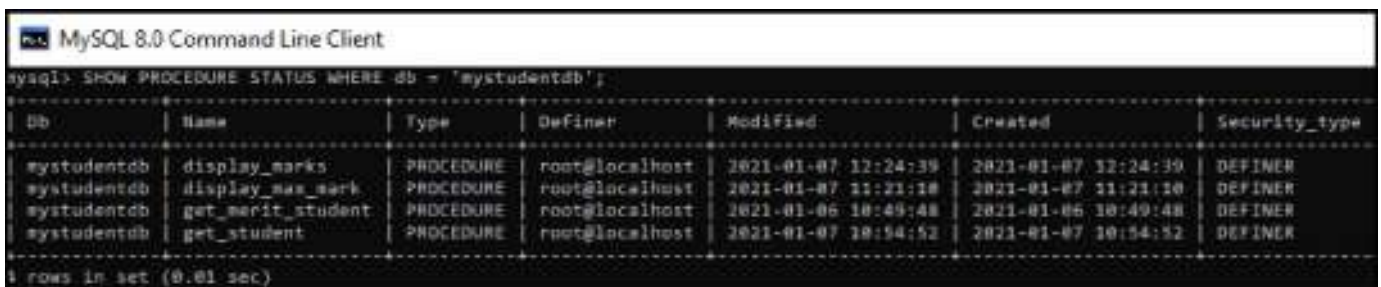
1. SHOW **PROCEDURE** STATUS [LIKE 'pattern' | **WHERE** search_condition]

This statement displays all stored procedure names, including their characteristics. If we want to display procedures in a particular database, we need to use the **WHERE clause**. In case we want to list stored procedures with a specific word, we need to use the **LIKE clause**.

We can list all stored procedure in the MySQL **mystudentsb** database using the below statement:

1. mysql> SHOW **PROCEDURE** STATUS **WHERE** db = 'mystudentdb';

It will give the below output where we can see that the mystudentdb database contains **four stored procedures**:



Db	Name	Type	Definer	Modified	Created	Security_type
mystudentdb	display_marks	PROCEDURE	root@localhost	2021-01-07 12:24:39	2021-01-07 12:24:39	DEFINER
mystudentdb	display_max_mark	PROCEDURE	root@localhost	2021-01-07 11:21:10	2021-01-07 11:21:10	DEFINER
mystudentdb	get_merit_student	PROCEDURE	root@localhost	2021-01-06 10:49:48	2021-01-06 10:49:48	DEFINER
mystudentdb	get_student	PROCEDURE	root@localhost	2021-01-07 10:54:52	2021-01-07 10:54:52	DEFINER

4 rows in set (0.01 sec)

How to delete/drop stored procedures in MySQL?

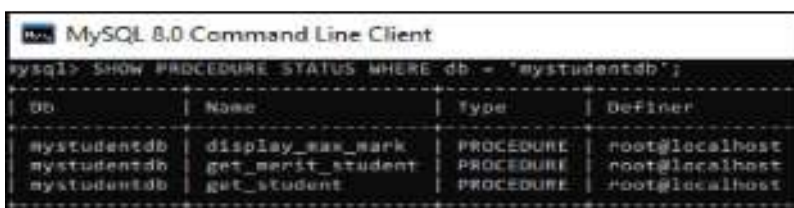
MySQL also allows a command to drop the procedure. When the procedure is dropped, it is removed from the database server also. The following statement is used to drop a stored procedure in MySQL:

1. **DROP PROCEDURE** [IF EXISTS] procedure_name;

Suppose we want to remove the procedure named **display_marks** from the mystudentdb database. We can do this by first selecting the database and then use the syntax as follows to remove the procedure:

1. mysql> **DROP PROCEDURE** display_marks;

We can verify it by listing the procedure in the specified database using the **SHOW PROCEDURE STATUS** command. See the below output:

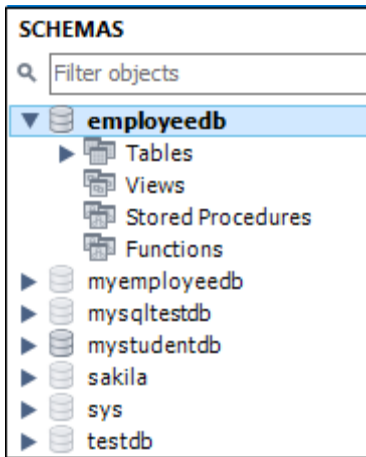


Db	Name	Type	Definer
mystudentdb	display_max_mark	PROCEDURE	root@localhost
mystudentdb	get_merit_student	PROCEDURE	root@localhost
mystudentdb	get_student	PROCEDURE	root@localhost

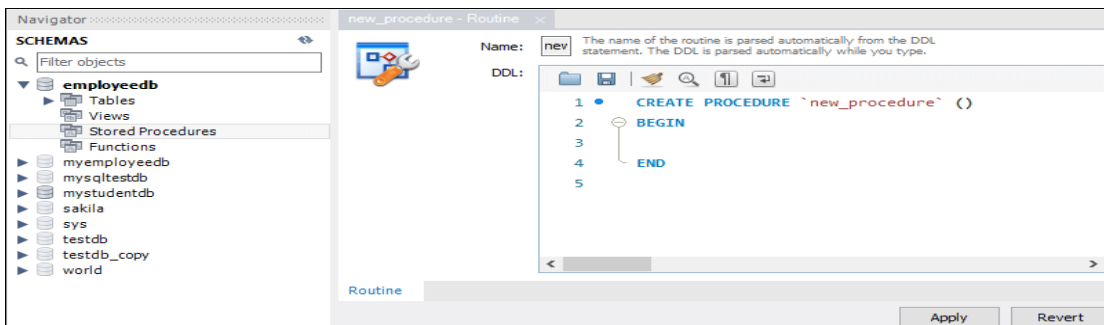
How to create a procedure in MySQL workbench?

We first launch the tool and log in with the username and password to create procedures in the [MySQL Workbench](#). Now, we need to do the following steps for creating a stored procedure:

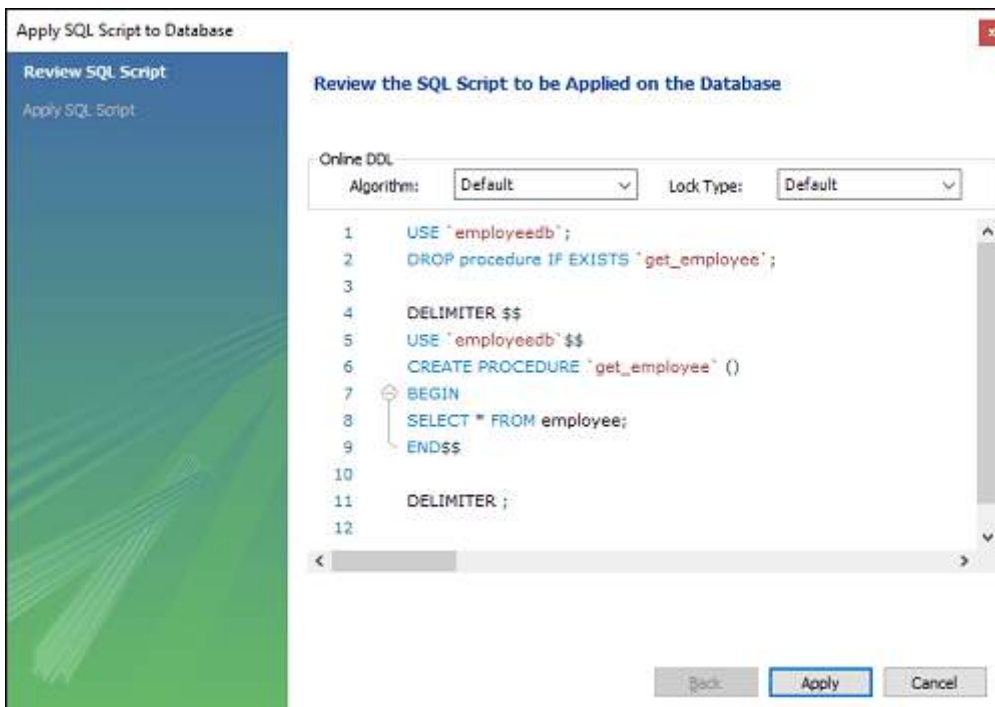
1. Go to the **Navigation tab** and click on the **Schema menu** where all the previously created databases available. Select your desired database (for example, **employeedb**). It will pop up the following options.



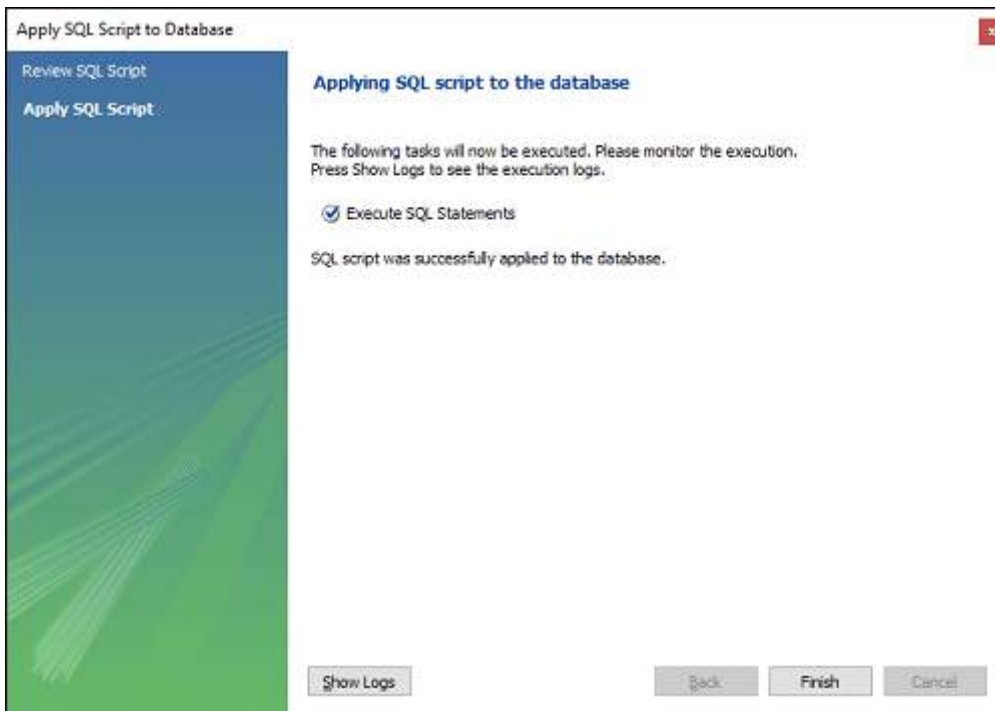
2. Right-click on the **Stored Procedure**, and we will get the default procedure code. See the below screen:



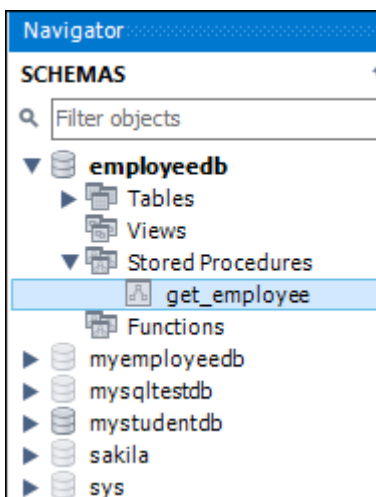
3. Complete the procedure code and click on the **Apply button**. In the next window, we will review the procedure code once again, and if no error was found, click on the Apply button.



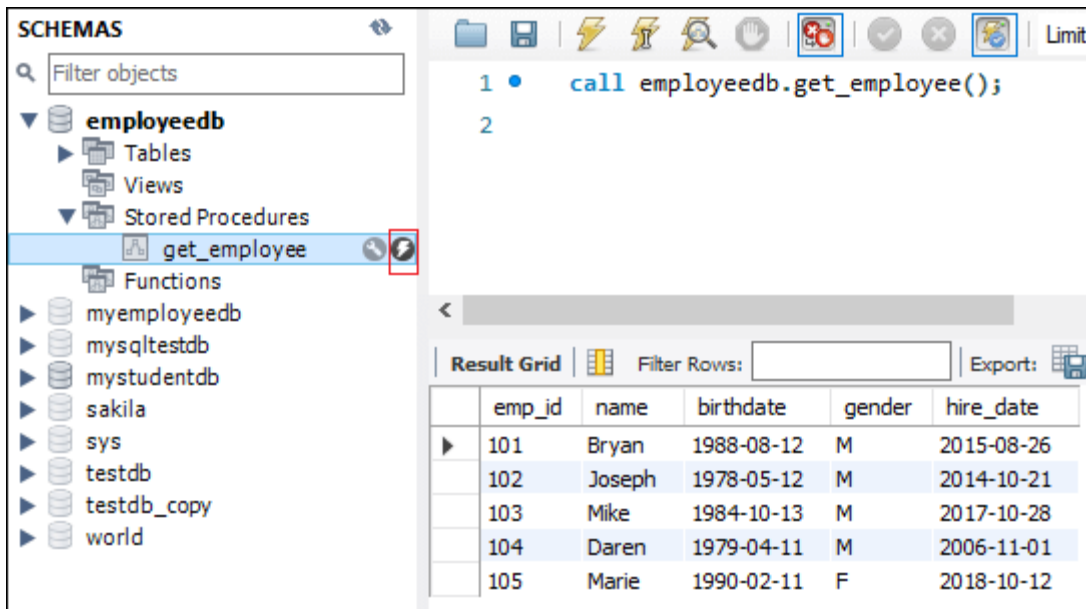
4. After clicking on the Apply button, click on the **Finish** button for completion.



5. We can navigate to the schema menu again to verify this newly created procedure. It means first select your database and expand it to display its sub-menu. In the **sub-menu**, expanding the stored procedure option will show the newly created procedure. See the below image:



6. We can call the procedure by tapping on the red rectangle box or simply execute the CALL statement.



How to alter the procedure in MySQL?

MySQL does not allow any command to alter the procedure in MySQL. However, it provides a command that is used to change the characteristics of a stored procedure. This command may alter more than one change in the procedure but does not modify the stored procedure's parameters or body. If we want to make such changes, we must **drop and re-create the procedure** using the DROP PROCEDURE and CREATE PROCEDURE statement.

The below statement is used to change the characteristics of a procedure but not the actual procedure:

1. **ALTER PROCEDURE** procedure_name [characteristics ...]
- 2.
3. characteristics: {
4. COMMENT 'string'
5. | LANGUAGE SQL
6. | { **CONTAINS** SQL | **NO** SQL | READS SQL DATA | MODIFIES SQL DATA }
7. | SQL SECURITY { DEFINER | INVOKER }
8. }

Suppose we want to **add a comment** to the existing procedure. In such a case, we can use the **ALTER** statement as follows to accomplish this task:

1. mysql> **ALTER PROCEDURE** get_merit_student
 2. COMMENT 'It displays all records';
- After executing this statement, we can verify it by using the below statement:

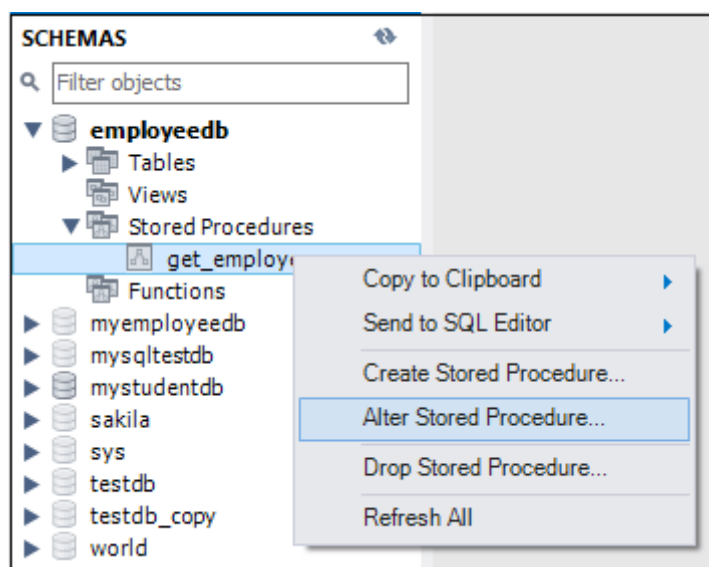
1. mysql> SHOW **CREATE PROCEDURE** get_merit_student \G;
- It will display the below output where we can see that the **comment** is added successfully.


```
MySQL 8.0 Command Line Client

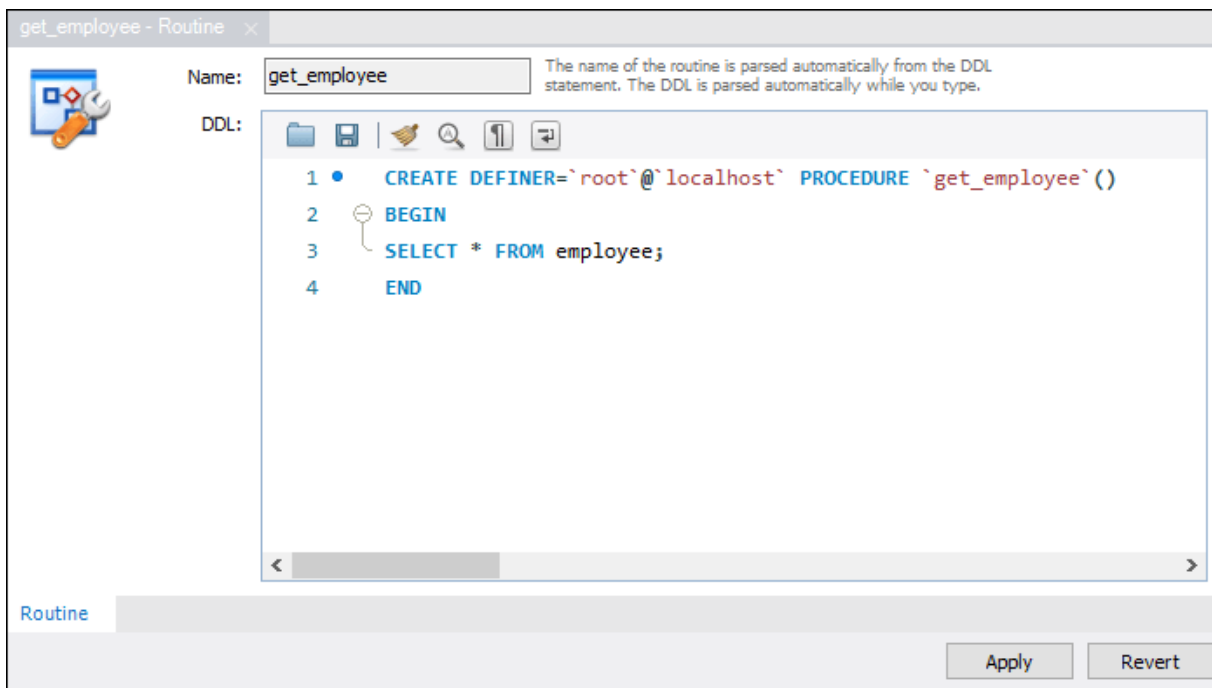
mysql> ALTER PROCEDURE get_merit_student
-> COMMENT 'It displays all records';
Query OK, 0 rows affected (0.12 sec)

mysql> SHOW CREATE PROCEDURE get_merit_student \G;
***** 1. row *****
      Procedure: get_merit_student
      sql_mode: STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION
      Create Procedure: CREATE DEFINER=`root`@`localhost` PROCEDURE `get_merit_student`()
      COMMENT 'It displays all records'
BEGIN
SELECT * FROM student_info WHERE marks > 70;
SELECT COUNT(stud_code) AS Total_Student FROM student_info;
END
character_set_client: cp850
collation_connection: cp850_general_ci
Database Collation: utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

It is to note that **we can alter the body of the stored procedure in MySQL using the workbench tool**. So open this tool, navigate to the schema menu, and expand the database that contains stored procedures. Now, select your procedure, right-click on it and choose **ALTER STORED PROCEDURE** option. See the below screen:

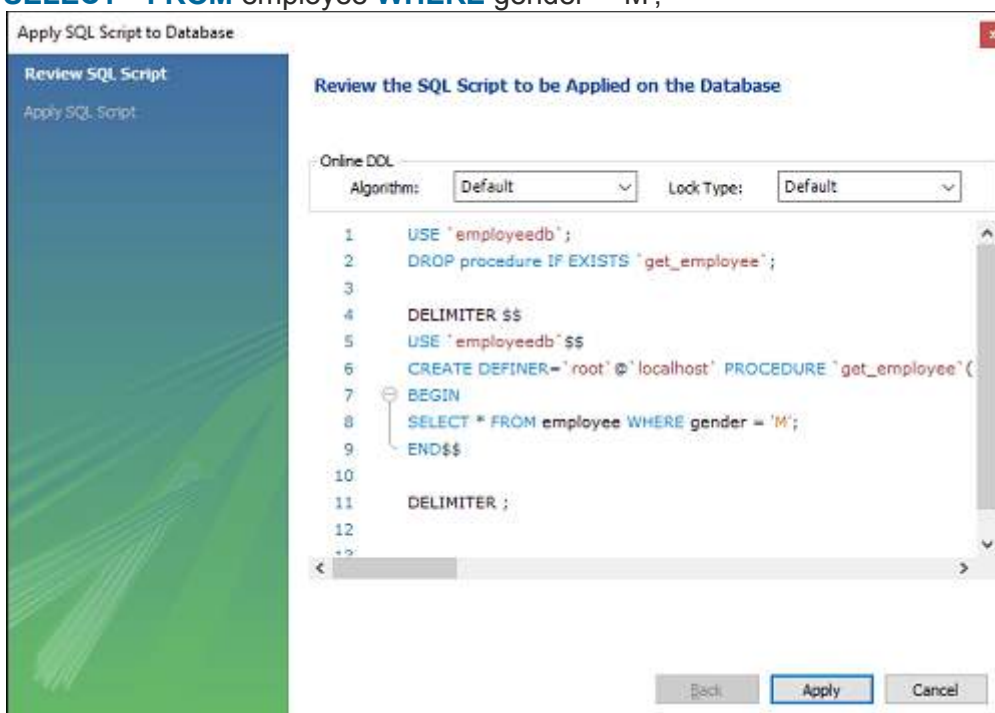


After clicking this option, we will get a window that contains a procedure code. See the below screen that contains procedure code to display all employee:



Now, we will modify this code. Suppose we want to display only male employees. To do this, we can change this code from the below code and click on the **Apply** button:

1. **SELECT * FROM** employee **WHERE** gender = 'M';



In this window, we will review the procedure code once again, and if no error was found, click on the **Apply -> Apply -> Finish** button to complete the process.

Drawbacks of Using Stored Procedures

- If we use stored procedures, the memory uses of every connection that uses those stored procedures will increase substantially. Also, if we overuse many logical applications inside stored procedures, the CPU usage will increase. It is because the database server is not well designed for logical operations.
- Stored procedure's constructs are not designed to develop complex and flexible business logic.

- It is difficult to debug stored procedures. Only a few database management systems allow us to debug stored procedures. Unfortunately, MySQL does not provide facilities for debugging stored procedures.
- It is not easy to develop and maintain stored procedures. Developing and maintaining stored procedures are often required a specialized skill set that not all application developers possess. It may lead to problems in both application development and maintenance phases.

MySQL Stored Function

A stored function in MySQL is a set of SQL statements that perform some task/operation and return a single value. It is one of the types of stored programs in MySQL. When you will create a stored function, make sure that you have a CREATE ROUTINE database privilege. Generally, we used this function to encapsulate the common business rules or formulas reusable in stored programs or SQL statements.

The stored function is almost similar to the procedure in MySQL, but it has some differences that are as follows:

- The function parameter may contain only the IN parameter but can't allow specifying this parameter, while the procedure can allow IN, OUT, INOUT parameters.
- The stored function can return only a single value defined in the function header.
- The stored function may also be called within SQL statements.
- It may not produce a result set.

Thus, we will consider the stored function when our program's purpose is to compute and return a single value only or create a user-defined function.

The syntax of creating a stored function in MySQL is as follows:

1. DELIMITER \$\$
- 2.
3. CREATE FUNCTION fun_name(fun_parameter(s))
4. RETURNS datatype
5. [NOT] {Characteristics}
6. fun_body;

Parameter Used

The stored function syntax uses the following parameters which are discussed below:

Parameter Name	Descriptions
fun_name	It is the name of the stored function that we want to create in a database. It should not be the same as the built-in function name of MySQL.

fun_parameter	It contains the list of parameters used by the function body. It does not allow to specify IN, OUT, INOUT parameters.
datatype	It is a data type of return value of the function. It should any valid MySQL data type.
characteristics	The CREATE FUNCTION statement only accepted when the characteristics (DETERMINISTIC, NO SQL, or READS SQL DATA) are defined in the declaration.
fun_body	This parameter has a set of SQL statements to perform the operations. It requires at least one RETURN statement. When the return statement is executed, the function will be terminated automatically. The function body is given below: BEGIN -- SQL statements END \$\$ DELIMITER

MySQL Stored Function Example

Let us understand how stored function works in MySQL through the example. Suppose our database has a table named "customer" that contains the following data:

cust_id	name	occupation	age
101	Peter	Engineer	32
102	Joseph	Developer	30
103	John	Leader	28
104	Stephen	Scientist	45
105	Suzi	Carpenter	26
106	Bob	Actor	25

Now, we will create a function that returns the customer occupation based on the age using the below statement.

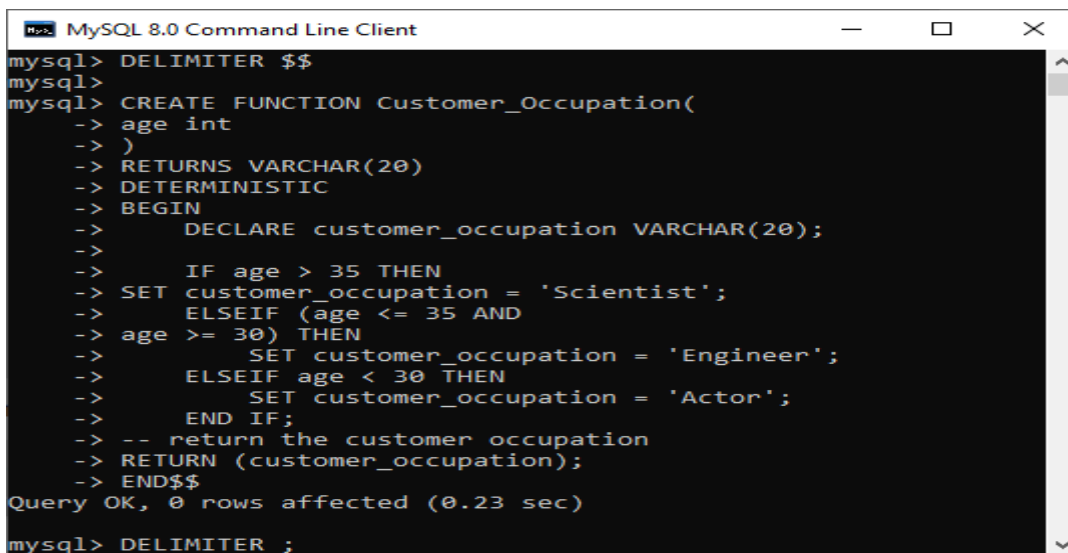
1. DELIMITER \$\$
2. CREATE FUNCTION Customer_Occupation(
3. age int
4.)
5. RETURNS VARCHAR(20)
6. DETERMINISTIC

```

7. BEGIN
8. DECLARE customer_occupation VARCHAR(20);
9. IF age > 35 THEN
10. SET customer_occupation = 'Scientist';
11. ELSEIF (age <= 35 AND
12. age >= 30) THEN
13. SET customer_occupation = 'Engineer';
14. ELSEIF age < 30 THEN
15. SET customer_occupation = 'Actor';
16. END IF;
17. -- return the customer occupation
18. RETURN (customer_occupation);
19. END$$
20. DELIMITER;

```

Execute the above statement on the command-line tool, as shown below:

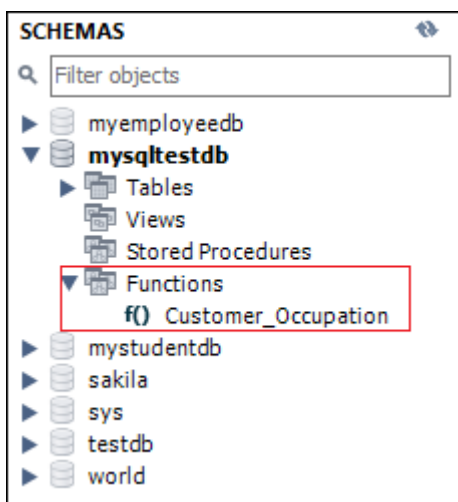


```

MySQL 8.0 Command Line Client
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION Customer_Occupation(
-> age int
-> )
-> RETURNS VARCHAR(20)
-> DETERMINISTIC
-> BEGIN
-> DECLARE customer_occupation VARCHAR(20);
->
-> IF age > 35 THEN
-> SET customer_occupation = 'Scientist';
-> ELSEIF (age <= 35 AND
-> age >= 30) THEN
-> SET customer_occupation = 'Engineer';
-> ELSEIF age < 30 THEN
-> SET customer_occupation = 'Actor';
-> END IF;
-> -- return the customer occupation
-> RETURN (customer_occupation);
-> END$$
Query OK, 0 rows affected (0.23 sec)
mysql> DELIMITER ;

```

Once the function creation is successful, we can see it in the MySQL workbench under the Function section like below image:



We can also see all stored functions available in the current database using the following statement:

1. SHOW FUNCTION STATUS WHERE db = 'mysqltestdb';

After executing the above command, we will get the output as below:

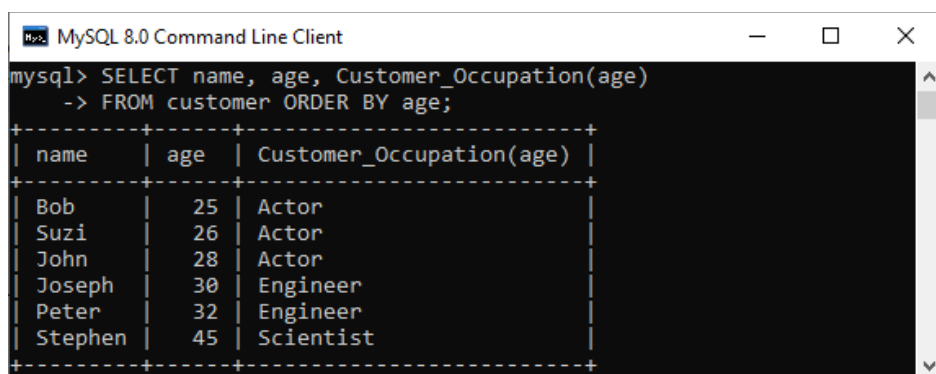
Db	Name	Type	Definer
mysqltestdb	Customer_Occupation	FUNCTION	root@localhost

Stored Function Call

Now, we are going to see how stored function is called with the SQL statement. The following statement uses customer_occupation stored function to get the result:

1. SELECT name, age, Customer_Occupation(age)
2. FROM customer ORDER BY age;

It will give the output as below.



```
mysql> SELECT name, age, Customer_Occupation(age)
-> FROM customer ORDER BY age;
```

name	age	Customer_Occupation(age)
Bob	25	Actor
Suzi	26	Actor
John	28	Actor
Joseph	30	Engineer
Peter	32	Engineer
Stephen	45	Scientist

Stored Function Call in Procedure

Here, we are going to see how this function can be called in a stored procedure. This statement creates a procedure in a database that uses Customer_Occupation() stored function.

1. DELIMITER \$\$
2. CREATE PROCEDURE GetCustomerDetail()
3. BEGIN
4. SELECT name, age, Customer_Occupation(age) FROM customer ORDER BY age;
5. END\$\$
6. DELIMITER ;

The below statement can be used to call the stored procedure:

1. CALL GetCustomerDetail();

We will get the output as below:

```
MySQL 8.0 Command Line Client
mysql> CALL GetCustomerDetail(); \G
***** 1. row *****
      name: Bob
      age: 25
Customer_Occupation(age): Actor
***** 2. row *****
      name: Suzi
      age: 26
Customer_Occupation(age): Actor
***** 3. row *****
      name: John
      age: 28
Customer_Occupation(age): Actor
***** 4. row *****
      name: Joseph
      age: 30
Customer_Occupation(age): Engineer
***** 5. row *****
      name: Peter
      age: 32
Customer_Occupation(age): Engineer
***** 6. row *****
      name: Stephen
      age: 45
Customer_Occupation(age): Scientist
```

Unit-5.0: Database Connection and

Error handling

5.1 Connect database

5.1.1 connect using mysqli(), pdo()

5.1.2 advantage of pdo over mysqli

5.2 Data Read Write using PDO

5.2.1 prepare(), execute()

5.2.2 fetch() with fetchassoc

5.2.3 beginTransaction(),

commit(), rollback()

5.3 Error handling

5.3.1 try, catch

MySQL Connection

A connection is a computer science facility that allows the user to connect with the database server software. A user can connect with the database server, whether on the same machine or remote locations. Therefore, if we want to work with the database server to send commands and receive answers in the form of a result set, we need connections.

PHP MySQL Connect

Since PHP 5.5, `mysql_connect()` extension is *deprecated*. Now it is recommended to use one of the 2 alternatives.

- `mysqli_connect()`
 - `PDO::__construct()`
-

PHP `mysqli_connect()` Function:-

The `mysqli_connect()` function in PHP is a fundamental tool for establishing a connection to a MySQL database. This function is crucial for PHP applications that need to interact with MySQL databases, enabling them to execute queries, retrieve data, and perform various database operations.

In this article, We will learn about the PHP `mysqli_connect()` Function in detail by understanding various examples and so on.

PHP `mysqli_connect()` Function

- The `mysqli_connect()` function in PHP is used to establish a connection to a MySQL database. This function initiates a connection to the MySQL server and returns a connection object on success, or `FALSE` failure.
- It allows PHP scripts to interact with a MySQL database to perform various operations like querying, inserting, updating, and deleting data.

Syntax:

```
mysqli_connect ( "host", "username", "password", "database_name" );
```

- `host`: The hostname or IP address of the MySQL server.
- `username`: The username to access the MySQL server.
- `password`: The password associated with the given username.

- **database (optional):** The name of the database to select upon connection.
- **port (optional):** The port number to connect to the MySQL server. Default is 3306.
- **socket (optional):** The socket or named pipe to use for the connection.

Parameter Values

- **host:** A string representing the MySQL server's address.
- **username:** A string representing the username for MySQL authentication.
- **password:** A string representing the password for MySQL authentication.
- **database:** A string representing the database name to use (optional).
- **port:** An integer representing the port number (optional).
- **socket:** A string representing the socket or pipe for the connection (optional).

Return Value:

- It returns an object which represents the MySQL connection. If the connection failed then it returns FALSE.

Program: Below is the implementation of `mysqli_connect()` function.

```
<?php

mysqli_connect("localhost", "root", "", "GFG");

if (mysqli_connect_error()) {
    echo "Connection Error.";
} else {
    echo "Database Connection Successfully.";
}
?>
```

Output:

Database Connection Successfully.

PHP `mysqli_close()`

This function is used to disconnect with MySQL database. It returns *true* if connection is closed or *false*.

Syntax:- `bool mysqli_close(resource $resource_link)`

PDO in PHP

```
<?php
try {
    $dbhost = 'localhost';
    $dbname='gfg';
    $dbuser = 'root';
    $dbpass = '';
    $connect = new PDO("mysql:host=$dbhost;dbname=$dbname",
```

```

$dbuser, $dbpass);
    }
    catch (PDOException $e) {
        echo "Error : " . $e->getMessage() . "<br/>";
        die();
    }
    $count = $connect->exec("DELETE FROM employee WHERE emp_id
= 123 ");
    echo("Number of rows to be deleted from the employee table
is: ". $count);
?>

```

Connecting to a MySQL database server.

- MySQLi procedural
- MySQLi object-oriented
- PDO (PHP Data Objects)

MySQLi procedural and MySQLi object-oriented only support MySQL database but PDO is an advanced method along with MySQL which supports Postgres, SQLite, Oracle, and MS SQL Server. PDO is more secure than the first two options and it is also faster in comparison with MySQLi procedural and MySQLi object-oriented.

PDO is a database access layer that provides a fast and consistent interface for accessing and managing databases in PHP applications. Every DBMS has a specific PDO driver that must be installed when you are using PDO in PHP applications.

It simplifies the database operations including:

- Creating database connection
- Executing queries
- Handling errors
- Closing the database connections

When we want to connect PHP and MySQL then we need to follow 3 steps:

- Connection with the database
- Run SQL Query
- Closing the database connection

Steps for connection using PDO.

Connection:

In this step, we connect with the database

```
$conn = new PDO($db_name,$username,$password);
```

When we want the connection between PHP and MySQL using PDO then we need to create an object of the PDO class.

```
$db_name = "mysql:host=localhost;dbname:gfg";
```

Note: \$db_name contains the following information.

Run SQL query: To run SQL query we can use two methods:

- query():

```
$sql = $conn->query("select * from students");
```


- prepare():

```
$sql = $conn->prepare("select * from students");
```

Note: The prepare() method is more secure than the query() method.

Close the connection:

For closing the database connection using PDO.

```
$conn = null;
```

Parameters: It contains the following parameters.

- dsn: It contains the information regarding connection to the database.
- user: It contains the user name.
- password: It contains the password of the database.

Return Value:

On success, it will return the PDO object. And on failure, it will return the PDOException object.

Handling error during connection:

The PDOException object will be thrown in case of any error. We can catch the exception to handle the error.

• PHP

```
<?php
    try {
        $dsn = "mysql:host=localhost;dbname=gfg";
        $user = "root";
        $passwd = "";
        $pdo = new PDO($dsn, $user, $passwd);
    }
    catch (PDOException $e) {
        echo "Error!: " . $e->getMessage() . "<br/>";
        die();
    }
?>
```

Output:

```
Error : SQLSTATE[28000] [1045]
```

```
Access denied for user 'root'@'localhost'
```

```
(using password: NO)
```

Example 1: Suppose we have a database “gfg” with a table “students”. We want to fetch the details like “id” and name of all the students present in the table “students”.

• PHP

```
<?php
    $dsn = "mysql:host=localhost;dbname=gfg";
    $user = "root";
    $passwd = "";
```

```

$pdo = new PDO($dsn, $user, $passwd);
$stmt = $pdo->query("SELECT * FROM students");
$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
foreach($rows as $row) {
    printf("{ $row['id']} { $row['name'] }\n");
}
?>

```

Output:

```

1 Student1
2 Student2
3 Student3
4 Student4

```

Example 2: In this case, suppose we have the “employee” table in “gfg” database. We want to delete the employee from the “employee” table whose “id = 123”.

- PHP

```

<?php
    try {
        $dbhost = 'localhost';
        $dbname = 'gfg';
        $dbuser = 'root';
        $dbpass = '';
        $connect = new PDO(
            "mysql:host=$dbhost;dbname=$dbname", $dbuser, $dbpass);
    }
    catch (PDOException $e) {
        echo "Error : " . $e->getMessage() . "<br/>";
        die();
    }
    $count = $connect->exec("DELETE FROM employee WHERE emp_id
= 123 ");
    echo("Number of rows to be deleted from the employee table
is: " . $count);
?>

```

Output:

```

The number of rows to be deleted from the employee table is: 1

```

Benefits of PDO:

- Usability: PDO contains helper functions to operate automatic routine operations.
- Reusability: We can access multiple databases because it offers a unified API.

- Security: It provides protection from SQL injection because it uses a prepared statement. A prepared statement separates the instruction of the SQL statement from the data.
- Error handling: It uses exceptions for error handling. There are three types of modes:
 - Silent
 - Warning
 - Exception
- Multiple database support: It is used to access any database which is written for the PDP driver. We need to find a suitable driver and add them when we use them. There are several PDO drivers available like Microsoft SQL Server, Sybase, PostgreSQL, and many more.

Supported Database: PDO supports 12 different databases.

- MySQL
- PostgreSQL
- Oracle
- Firebird
- MS SQL Server
- Sybase
- Informix
- IBM
- FreeTDS
- SQLite
- Cubrid
- 4D

Fetch with fetch_assoc()

The fetch_assoc() / mysqli_fetch_assoc() function fetches a result row as an associative array.

Note: Fieldnames returned from this function are case-sensitive.

Fetch a result row as an associative array:

```
<?php
$mysqli = new mysqli("localhost", "my_user", "my_password", "my_db");

if ($mysqli -> connect_errno) {
    echo "Failed to connect to MySQL: " . $mysqli -> connect_error;
    exit();
}

$sql = "SELECT Lastname, Age FROM Persons ORDER BY Lastname";
$result = $mysqli -> query($sql);

// Associative array
$row = $result -> fetch_assoc();
printf ("%s (%s)\n", $row["Lastname"], $row["Age"]);
```

```
// Free result set
$result -> free_result();

$mysqli -> close();
?>
```

MySQL Transaction

A transaction in MySQL is a **sequential group of statements**, queries, or operations such as select, insert, update or delete to perform as a one single work unit that can be committed or rolled back. If the transaction makes multiple modifications into the database, two things happen:

- Either all modification is successful when the transaction is committed.
- Or, all modifications are undone when the transaction is rollback.

In other words, a transaction cannot be successful without completing each operation available in the set. It means if any statement fails, the transaction operation cannot produce results.

A transaction in MySQL starts with the first executable SQL statement and ends when it finds a commit or rolled back either explicitly or implicitly. It explicitly uses COMMIT or ROLLBACK statement and implicitly when a DDL statement is used.

We can understand the concept of a transaction in [MySQL](#) by considering a **banking database**. Suppose a bank customer wants to transfer money from one account to another account. We can achieve this by using the SQL statements that will be divided into the following steps:

- First, it is required to check the availability of the requested amount in the first account.
- Next, if the amount is available, deduct it from the first account. Then, update the first account.
- Finally, deposit the amount in the second account. Then update the second account to complete the transaction.
- If any of the above processes fails, the transaction will be rolled back into its previous state.

Properties of Transaction

The transaction contains mainly four properties, which referred to as **ACID** property. Now, we are going to discuss the ACID property in detail. The ACID property stands for:

1. Atomicity
2. Consistency
3. Isolation
4. Durability

Atomicity: This property ensures that all statements or operations within the transaction unit must be executed successfully. Otherwise, if any operation is failed, the whole transaction will be aborted, and it goes rolled back into their previous state. It includes features:

- COMMIT statement.
- ROLLBACK statement.
- Auto-commit setting.
- Operational data from the INFORMATION_SCHEMA tables.

Consistency: This property ensures that the database changes state only when a transaction will be committed successfully. It is also responsible for protecting data from crashes. It includes features:

- InnoDB doublewrite buffer.
- InnoDB crash recovery.

Isolation: This property guarantees that each operation in the transaction unit operated independently. It also ensures that statements are transparent to each other. It includes features:

- SET ISOLATION LEVEL statement.
- Auto-commit setting.
- The low-level details of InnoDB locking.

Durability: This property guarantees that the result of committed transactions persists permanently even if the system crashes or failed. It includes features:

- Write buffer in a storage device.
- Battery-backed cache in a storage device.
- Configuration option `innodb_file_per_table`.
- Configuration option `innodb_flush_log_at_trx_commit`.
- Configuration option `sync_binlog`.

MySQL Transaction Statement

MySQL control transactions with the help of the following statement:

- MySQL provides a `START TRANSACTION` statement to begin the transaction. It also offers a `"BEGIN"` and `"BEGIN WORK"` as an alias of the `START TRANSACTION`.
- We will use a `COMMIT` statement to commit the current transaction. It allows the database to make changes permanently.
- We will use a `ROLLBACK` statement to roll back the current transaction. It allows the database to cancel all changes and goes into their previous state.
- We will use a `SET auto-commit` statement to disable/enable the auto-commit mode for the current transaction. By default, the `COMMIT` statement executed automatically. So if we do not want to commit changes automatically, use the below statement:

1. **SET** autocommit = 0;
2. OR,
3. **SET** autocommit = **OFF**:

Again, use the below statement to enable auto-commit mode:

1. **SET** autocommit = 1;
2. OR,
3. **SET** autocommit = **ON**:

MySQL Transaction Example

Suppose we have two tables named **"employees"** and **"Orders"** that contains the following data:

Table: employees

emp_id	emp_name	emp_age	city	income
101	Peter	32	Newyork	200000
102	Mark	32	California	300000
103	Donald	40	Arizona	1000000
104	Obama	35	Florida	5000000
105	Linklon	32	Georgia	250000
106	Kane	45	Alaska	450000
107	Adam	35	California	5000000
108	Macculam	40	Florida	350000
109	Brayan	32	Alaska	400000
110	Stephen	40	Arizona	600000

Table: orders

Order_ID	Product_Name	Order_Num	Order_Date
1	Laptop	5544	2020-02-01
2	Mouse	3322	2020-02-11
3	Desktop	2135	2020-01-05
4	Mobile	3432	2020-02-22
5	Anti-Virus	5648	2020-03-10

COMMIT Example

If we want to use a transaction, it is required to break the [SQL](#) statements into logical portions. After that, we can define whether the data should be committed or rollback.

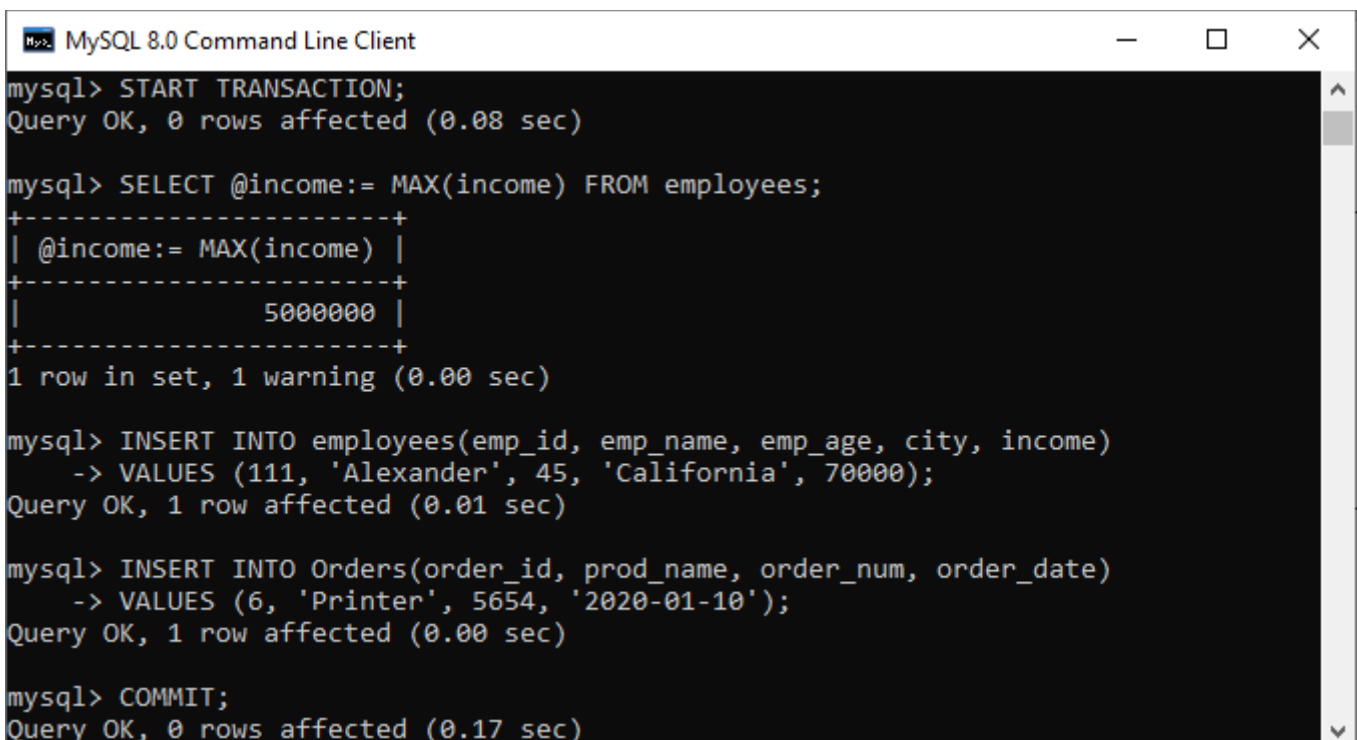
The following steps illustrate to create a transaction:

1. Begin the transaction using the START TRANSACTION statement.
2. Then, select maximum income among the employee.
3. Add a new record to the employee table.
4. Add a new record into the order table.
5. Use the COMMIT statement to complete the transaction.

Below are the commands that perform the above operations:

1. -- 1. Start a new transaction
2. **START TRANSACTION**;
3. -- 2. Get the highest income
4. **SELECT** @income:= **MAX**(income) **FROM** employees;
5. -- 3. Insert a new record into the employee table
6. **INSERT INTO** employees(emp_id, emp_name, emp_age, city, income)
7. **VALUES** (111, 'Alexander', 45, 'California', 70000);
8. -- 4. Insert a new record into the order table
9. **INSERT INTO** Orders(order_id, prod_name, order_num, order_date)
10. **VALUES** (6, 'Printer', 5654, '2020-01-10');
11. -- 5. Commit changes
12. **COMMIT**;

The below image explains it more clearly:



```

MySQL 8.0 Command Line Client
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.08 sec)

mysql> SELECT @income:= MAX(income) FROM employees;
+-----+
| @income:= MAX(income) |
+-----+
|          5000000      |
+-----+
1 row in set, 1 warning (0.00 sec)

mysql> INSERT INTO employees(emp_id, emp_name, emp_age, city, income)
-> VALUES (111, 'Alexander', 45, 'California', 70000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Orders(order_id, prod_name, order_num, order_date)
-> VALUES (6, 'Printer', 5654, '2020-01-10');
Query OK, 1 row affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.17 sec)

```

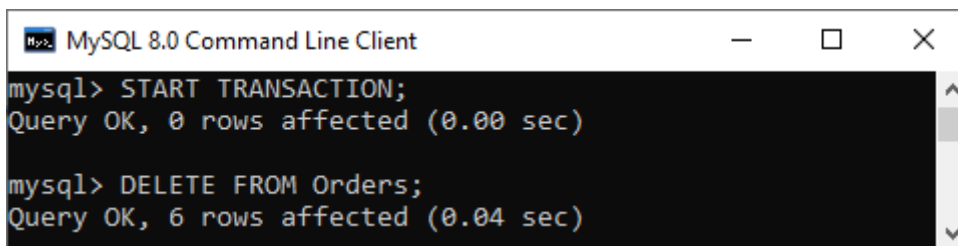
ROLLBACK Example

We can understand the rollback transaction with the help of the following illustration. First, open the MySQL command prompt and log into the database server using the password. Next, we have to select a database.

Suppose our database contains the "**Orders**" table. Now, the following are the scripts that perform the rollback operations:

1. -- 1. Start a new transaction
2. **START TRANSACTION**;
- 3.
4. -- 2. Delete data from the order table
5. **DELETE FROM** Orders;

After the execution of the above statement, we will get the output as below that shows all the records from the table Orders were successfully deleted.

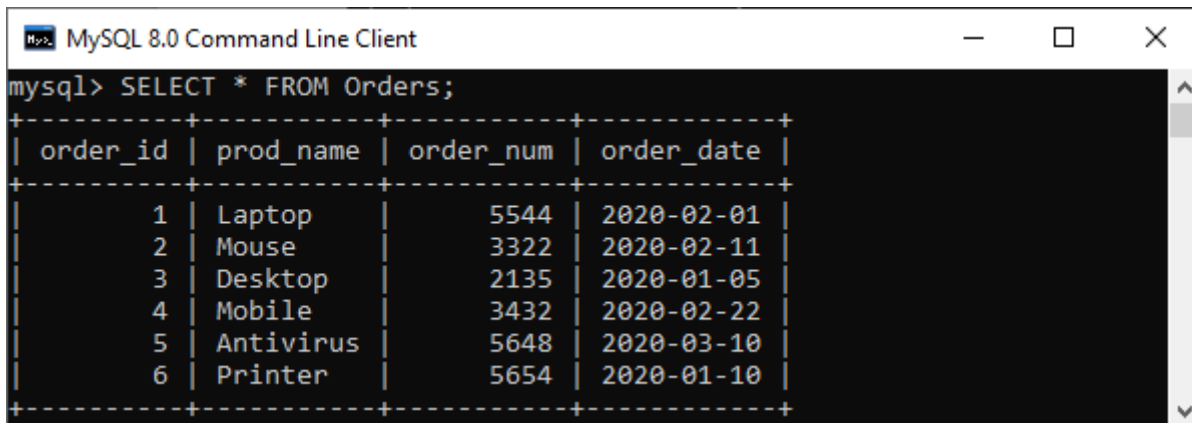


```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> DELETE FROM Orders;
Query OK, 6 rows affected (0.04 sec)
```

Now, we need to open a separate session of MySQL database server and execute the below statement to verify the data in Orders table:

1. **SELECT * FROM** Orders;
- It will give the output as below.



```
mysql> SELECT * FROM Orders;
+-----+-----+-----+-----+
| order_id | prod_name | order_num | order_date |
+-----+-----+-----+-----+
| 1 | Laptop | 5544 | 2020-02-01 |
| 2 | Mouse | 3322 | 2020-02-11 |
| 3 | Desktop | 2135 | 2020-01-05 |
| 4 | Mobile | 3432 | 2020-02-22 |
| 5 | Antivirus | 5648 | 2020-03-10 |
| 6 | Printer | 5654 | 2020-01-10 |
+-----+-----+-----+-----+
```

Although we have made changes in the first session, we still can see the records are available in the table. It is because the changes are not permanent until we have not executed the **COMMIT** or **ROLLBACK** statement in the first session.

Therefore if we want to make changes permanent, use the COMMIT statement. Otherwise, execute the ROLLBACK statement to roll back the changes in the first session.

1. -- 3. Rollback changes
2. **ROLLBACK**;
- 3.
4. -- 4. Verify the records in the first session
5. **SELECT * FROM** Orders;

After the successful execution, it will produce the following result where we can see that the change has been rolled back.

```
MySQL 8.0 Command Line Client
mysql> ROLLBACK;
Query OK, 0 rows affected (1.68 sec)

mysql> SELECT * FROM Orders;
+-----+-----+-----+-----+
| order_id | prod_name | order_num | order_date |
+-----+-----+-----+-----+
| 1 | Laptop | 5544 | 2020-02-01 |
| 2 | Mouse | 3322 | 2020-02-11 |
| 3 | Desktop | 2135 | 2020-01-05 |
| 4 | Mobile | 3432 | 2020-02-22 |
| 5 | Antivirus | 5648 | 2020-03-10 |
| 6 | Printer | 5654 | 2020-01-10 |
+-----+-----+-----+-----+
```

Statements that cannot be a rollback in using MySQL Transaction.

MySQL Transaction cannot be able to roll back all statements. For example, these statements include DDL (Data Definition Language) commands such as CREATE, ALTER, or DROP database as well as CREATE, UPDATE, or DROP tables or stored routines. We have to make sure that when we design our transaction, these statements do not include.

SAVEPOINT, ROLLBACK TO SAVEPOINT, RELEASE SAVEPOINT

The **SAVEPOINT** statement creates a special mark with the name of the **identifier** inside a transaction. It allows all statements that are executed after savepoint would be rolled back. So that the transaction restores to the previous state it was in at the point of the savepoint. If we have set multiple savepoints in the current transaction with the same name, the newly savepoint is responsible for rollback.

The **ROLLBACK TO SAVEPOINT** statement allows us to rolls back all transactions to the given savepoint was established without aborting the transaction.

The **RELEASE SAVEPOINT** statement destroys the named savepoint from the current transaction without undoing the effects of queries executed after the savepoint was established. After these statements, no rollback command occurs. If the savepoint does not exist in the transaction, it gives an error.

The following are the **syntax** of the above statements in MySQL Transaction:

1. SAVEPOINT savepoint_name
2. **ROLLBACK TO** [SAVEPOINT] savepoint_name
3. RELEASE SAVEPOINT savepoint_name

Example

Let us understand how to use these statements through the example. In the below example, we are going to use SAVEPOINT and ROLLBACK TO SAVEPOINT statements that explain how a savepoint determines which records of the current transaction can be rolled back.

1. START **TRANSACTION**;
2. **SELECT * FROM** Orders;
3. **INSERT INTO** Orders(order_id, prod_name, order_num, order_date)
4. **VALUES** (6, 'Printer', 5654, '2020-01-10');
5. SAVEPOINT my_savepoint;
6. **INSERT INTO** Orders(order_id, prod_name, order_num, order_date)
7. **VALUES** (7, 'Ink', 5894, '2020-03-10');
8. **ROLLBACK TO** SAVEPOINT my_savepoint;

9. **INSERT INTO** Orders(order_id, prod_name, order_num, order_date)
10. **VALUES** (8, 'Speaker', 6065, '2020-02-18');
11. **COMMIT**;

In the above,

- We have to first begin the transaction and then show the records available in the Orders table.
- Next, we have inserted one record into the table and then creates a savepoint mark.
- Again, we have inserted one record into the table and then use a ROLLBACK TO SAVEPOINT statement to remove changes where the savepoint established.
- Again, we have inserted one record into the table.
- Finally, execute the COMMIT statement to make changes permanently.

The output below explains the above steps in a sequential order that helps to understand it very easily.

```
MySQL 8.0 Command Line Client
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM Orders;
+-----+-----+-----+-----+
| order_id | prod_name | order_num | order_date |
+-----+-----+-----+-----+
| 1 | Laptop | 5544 | 2020-02-01 |
| 2 | Mouse | 3322 | 2020-02-11 |
| 3 | Desktop | 2135 | 2020-01-05 |
| 4 | Mobile | 3432 | 2020-02-22 |
| 5 | Antivirus | 5648 | 2020-03-10 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> INSERT INTO Orders(order_id, prod_name, order_num, order_date)
-> VALUES (6, 'Printer', 5654, '2020-01-10');
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT my_savepoint;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO Orders(order_id, prod_name, order_num, order_date)
-> VALUES (7, 'Ink', 5894, '2020-03-10');
Query OK, 1 row affected (0.00 sec)

mysql> ROLLBACK TO SAVEPOINT my_savepoint;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO Orders(order_id, prod_name, order_num, order_date)
-> VALUES (8, 'Speaker', 6065, '2020-02-18');
Query OK, 1 row affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.21 sec)
```

Now, we will use a SELECT statement to verify the above operation. In the output, we can see that the **order_id=6** and **order_id=8** is added successfully, but **order_id=7** is not inserted into the table. It rolls back the values entered after the savepoint was established:

```
MySQL 8.0 Command Line Client
mysql> SELECT * FROM Orders;
+-----+-----+-----+-----+
| order_id | prod_name | order_num | order_date |
+-----+-----+-----+-----+
|      1 | Laptop   |      5544 | 2020-02-01 |
|      2 | Mouse    |      3322 | 2020-02-11 |
|      3 | Desktop  |      2135 | 2020-01-05 |
|      4 | Mobile   |      3432 | 2020-02-22 |
|      5 | Antivirus |     5648 | 2020-03-10 |
|      6 | Printer  |     5654 | 2020-01-10 |
|      8 | Speaker  |     6065 | 2020-02-18 |
+-----+-----+-----+-----+
```

Now we are going to take another example RELEASE SAVEPOINT that establishes the my_savepoint and then removes a savepoint.

1. **START TRANSACTION;**
2. **INSERT INTO** Orders(order_id, prod_name, order_num, order_date)
3. **VALUES** (7, 'Ink', 5894, '2020-03-10');
4. **SAVEPOINT** my_savepoint;
5. **UPDATE** Orders **SET** prod_name='Scanner' **WHERE** order_id=8;
6. **RELEASE SAVEPOINT** my_savepoint;
7. **COMMIT;**

In the output, we can see that all statements in the transaction executed successfully. Here, both INSERT and UPDATE statements modify the table at COMMIT.

```
MySQL 8.0 Command Line Client
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.25 sec)

mysql> INSERT INTO Orders(order_id, prod_name, order_num, order_date)
-> VALUES (7, 'Ink', 5894, '2020-03-10');
Query OK, 1 row affected (0.02 sec)

mysql> SAVEPOINT my_savepoint;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Orders SET prod_name='Scanner' WHERE order_id=8;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> RELEASE SAVEPOINT my_savepoint;
Query OK, 0 rows affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.17 sec)
```

Server Virtualisation: Server virtualization is a process of partitioning the resources of a single server into multiple virtual servers. These virtual servers can run as separate machines. Server virtualization allows businesses to run multiple independent tasks with different configurations using a single (host) server. The process also saves the hardware cost involved in keeping a host of physical servers.

Block and File level Storage Virtualisation: Storage virtualization is a series of servers that are managed by a virtual storage system. The servers aren't aware of exactly where their data is stored. This technology manages the storage of data from multiple users and utilized as a single storage system. storage virtualization software maintains smooth operations and consistent performance despite changes, break down and differences in the connected equipment.

Data virtualization: This is a kind of virtualization in which the data is collected from various sources and managed that in a single server without knowing more about the technical information like how data is collected, stored & formatted. Then the stored data can be arranged in such a way so that its virtual view can be accessed by interested users by using the various cloud services remotely. Many big giant companies are providing their services like Oracle, IBM, At scale, Cdata etc.

Desktop as a service (DaaS)

Desktop as a service (DaaS) is a cloud-based desktop virtualization service hosted by a third party enterprise. The third party cloud provider manages all back-end resources, such as desktop storage, compute, and networking, including the virtual cloud machines that run the desktop operating systems. The provider streams the virtual desktops to end-user devices, allowing anytime, anywhere access to desktops and applications. There are two kinds of desktops available in DaaS. These are:

1. **Persistent desktop:** Users have the ability to customize and save the desktop so that it will look the same way each time when a particular user logs in. Persistent desktop requires more storage than non-persistent desktop.
2. **Non-persistent desktop:** Desktops are wiped each time the user logs out—they are merely a way to access shared cloud services.

DaaS Advantages:

- Easy platform migration
- Total cost reduction
- Minimized complexity
- Disaster recovery
- Uninterrupted connectivity
- Increased performance
- Personalization
- Reliability
- Data security